Szent István Katolikus Technikum és Gimnázium

# DIVISUM

**SZOFTVERFEJLESZTÉS ÉS -TESZTELÉS**

**VIZSGAREMEK**


**Készítették:**

**ERDEI MÁTÉ**

**TÓBIÁS SZABOLCS**


Sátoraljaújhely, 2026

# Tartalomjegyzék

BACKEND ..... 5
1. Rövid összefoglaló ..... 5
2. Használati útmutató röviden ..... 5
3. Rendszerkövetelmények ..... 5
3.1 Minimális követelmények: ..... 5
3.2 Ajánlott (kedvezőbb) követelmények: ..... 6
4. Környezet és konfiguráció ..... 6
5. Diagramok és vizuális segédeszközök ..... 7
5.1 Ajánlott VS Code kiterjesztések ..... 7
6. Adatmodell tervezés (magas szint) ..... 7
6.1 Táblák és mezők (részletesen magyarázattal) ..... 11
6.2 Kapcsolatok (ERD alapok) ..... 13
7. Migrációk és seedelés ..... 13
8. Kontroller és route minta ..... 14
9. Hitelesítés és jogosultságkezelés ..... 15
10. Fájlfeltöltés és statikus fájlok ..... 15
11. Hibakezelés és naplózás ..... 16
12. Biztonsági ajánlások ..... 16
13. Backup és restore stratégia ..... 17
14. Tesztelés ..... 17
15. Telepítés és CI ..... 17
16. API végpontok részletes lista (példa) ..... 18
17. API Végpontok Teljes Lista ..... 18
Felhasználó (User) Végpontok - /api/auth ..... 18
Admin Felhasználó Végpontok - /api/auth/admin ..... 18
Termék (Termek) Végpontok - /api/termek ..... 19

Admin Termék Végpontok - /api/termek/admin ..... 19
Rendelés (Rendeles) Végpontok - /api/rendeles ..... 20
Admin Rendelés Végpontok - /api/rendeles/admin ..... 20
Foglalás (Booking) Végpontok - /api/foglalas ..... 21
Admin Foglalás Végpontok - /api/foglalas/admin ..... 21
Feltöltés (Upload) Végpontok ..... 22
Általános Hibaüzenetek ..... 22
Megjegyzések ..... 22
18. Adat szótár (példák) ..... 22
18.1 Projekt Technikai Szójegyzéke ..... 22
18.2 Összefoglaló táblázat ..... 33
19. Példa lekérdezések és tranzakciós minta (pseudokód) ..... 36
20. Üzemeltetési checklist ..... 37
21. Változtatások és javasolt fejlesztések ..... 37
22. Mellékletek ..... 37
23. Postman tesztesetek (tervezet) ..... 37
1. Felhasználó regisztráció ..... 38
2. Bejelentkezés ..... 38
3. Terméklista lekérdezése ..... 39
4. Rendelés lekérése ..... 40
5. Profil lekérése ..... 40
6.Profil törlése ..... 41
7.Profil módosítás ..... 41
24. Részletes telepítési útmutató (Backend és Adatbázis) ..... 42
25.1 Lokális fejlesztés — Windows (PowerShell) lépésről lépésre ..... 42
25.2 XAMPP telepítés: ..... 45
25.3 Adatbázis létrehozása és jogosultságok (PowerShell vagy XAMPP): ..... 47
Adatbázis létrehozása a GUI felületen: ..... 47
25.4 Projekt klónozása és függőségek telepítése: ..... 51
Környezeti változók létrehozása a .env-ben — javasolt példa: ..... 52
25.5 Adatbázis konfiguráció részletek (Sequelize példával) ..... 53
25.6 Migrációk, seedek és adatbázis inicializálás ..... 54

25. Backup és Restore (parancsok) ... 54
26. Hálózat, tűzfal és portok ... 55
27. Hibakeresés — gyakori problémák és megoldások ... 55
28. Naplózás és monitoring ... 55
29. Frissítések ... 55
30. Biztonsági mentés és helyreállítás tesztelése ... 55
31. Mellékletek és példa fájlok ... 55
32. Részletes környezeti változó-gyűjtemény ... 56
33. Záró megjegyzések ... 57
34. Végszó ... 57
FRONTEND ... 57
Divisum Frontend Dokumentáció ... 57
1. Bevezetés ... 57
2. Projektstruktúra ... 58
Konfigurációs és Rendszerfájlok ... 59
Globális Stílusok és Erőforrások ... 60
Alkalmazásmag és Útvonalválasztás ... 60
Oldal-komponensek (UI Modulok) ... 60
Szolgáltatások és Biztonság ... 61
3. Főbb Angular komponensek ... 61
3.1. App modul és routing ... 61
3.2. Oldal-komponensek részletesen ... 63
3.3. Szolgáltatások (Services) ... 72
4. Stílusok és assetek ... 74
5. Build és futtatás ... 76
6. Továbbfejlesztési javaslatok ... 77
35. Források és dokumentáció ... 79

# BACKEND

## 1. Rövid összefoglaló

Ez a dokumentáció a Divisum projekt Backend rétegét és annak az adatbázis-működés részleteit tartalmazza. Különös hangsúlyt kap a részletes telepítési útmutató mind a backendre, mind az adatbázisra (helyi gépen), a környezeti változók, migrációk, seedelés, biztonsági beállítások és üzemeltetési feladatok. Ezek mellett lesz szó részletesen a végpontokról, azoknak a lehetséges státusz kódjairól és funkcióikról.

## 2. Használati útmutató röviden

- Következő fejezetek: architektúra, struktúra, konfiguráció, adatmodellek, részletes install (üzembehelyezés), migrációk, seedelés, futtatás, backup/restore és tipikus hibamegoldások.
- Minden parancs Windows PowerShell, Visual Studio Code környezetre van megadva, ahol szükséges.

## 3. Rendszerkövetelmények

- **Operációs rendszer:** Windows 10/11, macOS
- **Node.js:** LTS kiadás (18.x vagy 20.x javasolt)
- **NPM** Csomagkezelő.
- **XAMPP/MySQL:** Szerver adatbázisa. (XAMPP ajánlott)
- **Git:** Klónozáshoz és verziókezeléshez.
- **Tárhely:** Néhány 100 MB a projektfájloknak plusz adatbázis-méret.
- Kiegészítők: Postman vagy hasonló API kliens a tesztesetek futtatásához.

*Ez a dokumentáció a projekt fejlesztéséhez készült, a jövőben lehet majd online is futtatni; a backendet localhoston futtasd, nincs szükség proxy- vagy éles környezeti beállításokra. A környezeti változók egyszerűen a fejlesztői gépre állíthatók.*

Rendszerkövetelmények hardware részletesebben:

### 3.1 Minimális követelmények:

- Processzor: 2-magos CPU (Intel i3 vagy AMD Ryzen 1)

- RAM: 2 GB, több nagyobb adatbázis esetén
- Merevlemez: HDD elegendő alapszintű fejlesztéshez, néhány 100 MB szabad hely
- Hálózat: standard sávszélesség, localhost fejlesztéshez interneten kívül is működik

## 3.2 Ajánlott (kedvezőbb) követelmények:

- Processzor: 4+ magos CPU (Intel i5/i7 vagy AMD Ryzen 3/5)
- RAM: 4-8 GB, több nagyobb adatbázis esetén
- Merevlemez: SSD használata jelentősen gyorsítja a telepítési és adatbázis-műveleteket
- Hálózat: a távoli adatbázis-csatlakozás teszteléshez VPN vagy SSH tunel
- Architektúra áttekintés

Az alkalmazás egy Node.js + Express keretrendszerű szerver, amely REST API-t szolgáltat az Angular frontend felé és adatot kap tőle. Az adatbázis relációs (MariaDB/XAMPP javasolt a XAMPP), a szerver futtatásakor mindig feltöltődik a seeder-ben található adatokkal (termékek, admin), ha azokat nem találja benne a szerver. A szerkezet réteges: konfigurációk, modellek (Sequelize), kontrollerek, routeok, middleware és util fájlok. A képek tárolására az uploads mappában van hely.

# 4. Környezet és konfiguráció

A konfigurációk környezeti változókon alapulnak. A .env fájlban kerülnek tárolásra a következők:

- DB_HOST, DB_PORT, DB_USER, DB_PASS, DB_NAME
- JWT_SECRET, JWT_EXPIRES
- MAIL_HOST, MAIL_PORT, MAIL_USER, MAIL_PASS
- PORT (a szerver portja)
- SERVER_ADMIN_PASSWORD (Admin jelszó)

A config/db.js fájl felelős az adatbázis-kapcsolat létrehozásáért (Sequelize konfiguráció). A biztonsági gyakorlat, hogy a jelszavak és hitelesítések soha nem kerülnek verziókezelésbe, hanem .env-ben tárolódnak és a .env.example-ben szerepel a szükséges kulcsok listája.

# 5. Diagramok és vizuális segédeszközök

A projekt gyökérkönyvtárában található a databaseRelations.erd fájl, amely az adatbázis ER-diagramját tartalmazza. Fontos megnyitni és áttekinteni a táblák, mezők és relációk vizuális ábrázolását a rendszer mélyebb megértéséhez. A diagramot bármilyen ERD-képes szerkesztővel (pl. dbdiagram.io, MySQL Workbench) megnyithatod.

## 5.1 Ajánlott VS Code kiterjesztések

Ezek a kiterjesztések a fejlesztést segítik, de nem generálnak kódot.

- **ESLint** – JavaScript/Node linting és szabályok
- **Prettier – Code formatter** – egyenletes kódformázás
- **Node.js Extension Pack** – hasznos Node-specifikus eszközök
- **Docker** – konténer menedzsment (ha szükséges)
- **SQLTools** – adatbázis-kapcsolatok és lekérdezés futtatás
- **Database Client** – MariaDB/MySQL GUI a VS Code-on belül
- **GitLens** – Git történet és annotációk
- **Path Intellisense** – útvonal kiegészítés
- **Postman** – API végpontok tesztelése
- **Bracket Pair Colorizer** – kód olvashatóság javítása
- **Auto Rename Tag** – HTML/JSX tagek átnevezése (ha van frontend része)
- **EditorConfig for VS Code** – közös formázási szabályok betartása
- **Markdown All in One** – dokumentációíráshoz

# 6. Adatmodell tervezés (magas szint)

A rendszer főbb adatbázis táblái és entitásai:

Táblák:

1. User: felhasználói adatok, belépési információk, szerepkörök
2. Termek (Product): termékek adatai, leírás, ár, készlet, kép
3. Rendelesek (Order): megrendelések, státusz, összeg, kapcsolódó felhasználó
4. RendelesTermekek (OrderItems): rendelésben szereplő termékek, mennyiség, ár
5. Foglalas (Booking): időpontfoglalások, szolgáltatás, időpont, státusz

Egyébb entitások:

- Relations: az táblák közötti kapcsolatokat definiáló segédmodell

A modellezés során fontos a normalizáció: a redundancia csökkentése és a kapcsolatok explicit megjelenítése. A tervezéskor érdemes legalább harmadik normálforma (3NF) szerint szervezni a táblákat, és a mezők típusait gondosan meghatározni. Az alábbiakban szerepel egy javasolt táblastruktúra röviden mezőkkel. A táblák létrehozásához például a következő SQL parancsokat használhatjuk (ezek tovább finomíthatók indexeléssel, constraint-ekkel):

```
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nev VARCHAR(255) NOT NULL UNIQUE,
  email VARCHAR(255) NOT NULL UNIQUE,
  jelszo VARCHAR(255) NOT NULL,
  foglaltsag ENUM(‘Fogyasztó’, ‘Borbély’, 'Admin') DEFAULT 'Fogyasztó',
  pfPicture VARCHAR(255)
);

CREATE TABLE termekek (
  id INT AUTO_INCREMENT PRIMARY KEY,
  termekNev VARCHAR(255) NOT NULL,
  megjegyzes VARCHAR(255),
  ar DECIMAL(10,2) NOT NULL DEFAULT 0.00,
  kepNeve VARCHAR(255),
  kategoria VARCHAR(255)
);
```

```
-- A rendelés és rendeles_termekek kapcsolatai
CREATE TABLE rendelesek (
   id INT(255) PRIMARY KEY AUTO_INCREMENT,
   vasarloNeve VARCHAR(75) NOT NULL,
   vasarloEmail VARCHAR(75) NOT NULL,
   telefonszam VARCHAR(12),
   iranyitoszam VARCHAR(4) NOT NULL,
   telepules VARCHAR(255) NOT NULL,
   szallitasiCim VARCHAR(255) NOT NULL,
   rendelesIdeje DATETIME NOT NULL,
   termekek JSON NOT NULL,
   ar INT(255) NOT NULL,
   userId INT,
   FOREIGN KEY (userId) REFERENCES users(id) ON UPDATE CASCADE ON
DELETE CASCADE
);

CREATE TABLE rendelesTermekek (
  id INT AUTO_INCREMENT PRIMARY KEY,
  rendeles_id INT NOT NULL,
  termek_id INT NOT NULL,
  mennyiseg INT NOT NULL,
  ar DECIMAL(10,2) NOT NULL,
  FOREIGN KEY(rendeles_id) REFERENCES Rendelesek(id) ON DELETE CASCADE,
  FOREIGN KEY(termek_id) REFERENCES Termekek(id) ON DELETE CASCADE
);
```

```
CREATE TABLE foglalasok (
  id INT(255) PRIMARY KEY AUTO_INCREMENT,
  vasarloNeve VARCHAR(75) NOT NULL,
  vasarloEmail VARCHAR(255) NOT NULL,
  vasarloTelefonszam VARCHAR(12) NOT NULL,
  idopont DATETIME NOT NULL,
  borbely VARCHAR(75) NOT NULL,
  szolgaltatas VARCHAR(255) NOT NULL,
  ar INT(255) NOT NULL,
  userId INT,
  borbelyId INT,
  FOREIGN KEY (userId) REFERENCES users(id) ON UPDATE CASCADE ON DELETE SET NULL,
  FOREIGN KEY (borbelyId) REFERENCES users(id) ON UPDATE CASCADE ON DELETE SET NULL
);
```

A fenti kód hosszabb megjegyzésekkel és variánsokkal kiegészíthető, például az ON DELETE SET NULL használatával vagy extra indexek definiálásával az olvasási teljesítmény javítása érdekében.

## 6.1 Táblák és mezők (részletesen magyarázattal)

User

- id (PK, INT, AUTO_INCREMENT) – a felhasználó egyedi azonosítója
- nev (VARCHAR, UNIQUE) – a felhasználó neve
- email (VARCHAR, UNIQUE) – a felhasználó email-je
- jelszo (VARCHAR) – a felhasználó jelszava, amelyet mentés előtt hash-eljük
- foglaltsag (ENUM: "Fogyasztó", “Borbély”, "Admin") – a felhasználó lehetséges foglaltságai
- pfPicture (VARCHAR) - profilkép fájlnév/URL

Termek

- id (PK) - a termék egyedi azonosítója
- termekNev (VARCHAR) – a termék neve
- megjegyzes (VARCHAR) – a termék leírása ha van
- ar (INTEGER) – a termék ára
- kepNeve (VARCHAR) – a termékhez feltöltött kép neve, amely áthaladt a megfelelő feldolgozáson (multer-en)
- kategoria (VARCHAR) – termék kategóriája

Rendelesek

- id (PK) – a rendelés egyedi
- user_id (FK -> user.id) – a rendeléshez tartozó felhasználó egyedi azonosítója
- vasarloNeve (VARCHAR) – a rendeléshez tartozó felhasználó neve
- vasarloEmail (VARCHAR) – a rendeléshez tartozó felhasználó email-je
- telefonszam (VARCHAR) – a rendeléshez tartozó felhasználó telefonszáma
- iranyitoszam (VARCHAR) - a rendeléshez tartozó felhasználó irányitószáma
- telepules (VARCHAR) – a rendeléshez tartozó felhasználó települése

- szallitasiCim (VARCHAR) – rendeléshez tartozó felhasználó címe például utca, házszám
- rendelesIdeje (DATE) – a rendelés létre hozásának ideje
- ar (INTEGER) – a rendelés teljes ára
- termekek(JSON) – a rendelésben található termékek, mennyiséggel és árral.

RendelesTermekek

- id (PK) – egy rendelés termélének az egyedi azonosítója
- rendeles_id (FK -> rendelesek.id) – a rendelés egyedi azonosítója
- termek_id (FK -> termek.id) – a rendelés egy tárgyának az azonosítója
- mennyiseg (INTEGER) – a bizonyos termék mennyisége
- ar (INTEGER) – a termék árából és a mennyiségből adodó teljes ár

Foglalas

- id (PK) – a foglalás egyedi azonosítója
- user_id (FK -> user.id) – a foglaláshoz tartozó felhasználó egyedi azonosítója
- borbely_id (FK -> user.id where foglaltsag = “Borbély”) – a foglalásnál választott borbély egyedi azonosítója a Users táblából.
- idopont (DATETIME) – a foglalásnál megadott idő
- szolgaltatas (VARCHAR) – a foglalásnál megadott szolgáltatás például: hajvágás
- a (INTEGER) – a foglalásnál kiválasztott szolgáltatás ára
- vasarloNeve (VARCHAR) – a foglalásnál megadott fogyasztó neve
- vasarloEmail (VARCHAR) – a foglalásnál megadott fogysztó email címe
- vasarloTelefonszam (VARCHAR) – a foglalásnál megadott fogyasztó telefonszáma
- Borbely (VARCHAR) – a foglalásnál választott borbély neve

Relations

- id
- source_table
- source_id

- target_table
- target_id
- relation_type

## 6.2 Kapcsolatok (ERD alapok)

- User (1) → (N) Rendelesek
- Rendelesek (1) → (N) RendelesTermekek
- Termek (1) → (N) RendelesTermekek
- User (1) → (N) Foglalas

Ezen kapcsolatok indexekkel és külső kulcsokkal legyenek támogatva. A relációs integritás biztosítása érdekében használjuk az ON DELETE CASCADE vagy SET NULL szabályokat a domain igények szerint.

# 7. Migrációk és seedelés

A migrációk segítenek a séma verziókövetésében. Javasolt eszközök: Sequelize CLI, Umzug, vagy más migrációs keretrendszer. Minden változtatást migráción keresztül vezessünk be, hogy reprodukálható legyen a környezetek között.

Seederek: alapadmin felhasználó és minta, alap termékek hozzáadása (src/seeders/adminSeeder.js, termekSeeder.js). A seedelés csak fejlesztési vagy teszt környezetben fusson automatikusan; production esetén kézi jóváhagyással. Ha az adminok száma változik, ahoz képest be lehet írni vagy ki lehet venni sorokat a seeder-ből, ez ugyan így vonatkozik a termékekre. Ha egy admin vagy termék adatai változnak azt át KELL írni a seeder-ben, mert különben feltölti az eredetit a változott admin/termék mellé.

# 8. Kontroller és route minta

Minden erőforráshoz legyen külön route fájl és kontroller. Példa egy egyszerű termek végponthoz:

- GET /api/termekek -> lista
- GET /api/termekek/:id -> részletek
- POST /api/admin/termekCreate -> termék létrehozása (admin)
- PUT /api/admin/termekek/:id -> termék módosítása (admin)
- DELETE /api/admin/termekek/:id -> (admin)

**FONTOS**: Sok olyan végpont van, melyek arra lettek kialakítva, hogy csak az admin tudja használni rendszer biztonsági okokból például:

PUT /api/admin/termekek/:id -> termék módosítása (admin)

Ezeknél a végpontoknál **SOHA** nem lehet törölni az isAdmin() middleware-t, mert akkor egy egyszerű felhasználó simán beléphet az admin felületre és az egész weboldalt vagy túl terheli(például: létrehoz annyi felhasználót, hogy az adatbázis tárhely teljesen elfogy, létrehoz annyi terméket, hogy az oldal nem fogja tudni betölteni(RUNTIME ERROR)), esetleg olyan foglalást vagy rendelést hoz létre, amellyel más felhasználóknak, vagy az oldalnak okoz kárt vagy megadja magának az admin rangot és innentől ha nem veszik észre időben vagy egyáltalán nem veszik észre még javítás után is (Vissza írják az isAdmin() middleware-t, kijavítják) képes less admin-ként tevékenykedni, így hatalmas károkat tud okozni (előző esetek, amelyeket felsoroltam; másoknak ad nem megfelelő foglaltságot).

Kontrollerek és Route-ok: A kontroller a backend struktúrájában azt a réteget jelöli, ahol a bejövő kérések (a route-ok által átadott paraméterekkel, body-val stb.) feldolgozásra kerülnek, és meghatározzuk, milyen műveleteket kell végezni az adott erőforráson (adatbázis-lekérdezés, üzleti logika, válasz összeállítása). A route fájlok csak az útvonalak definiálásáért felelősek, a kontroller pedig olvashatóbb, újrahasználható funkciókat tartalmazz: például foglalás létrehozása, termékek listázása vagy felhasználó hitelesítése. Ezáltal az alkalmazás logikája jobban elkülönül, karbantarthatóbbá és tesztelhetőbbé válik, míg a routok egyszerűbbek és áttekinthetőbbek maradnak. Kontroller feladata: be- és kimeneti validáció, tranzakciók indítása (ha szükséges), hivatkozás a modellekre, hibakezelés, email küldés és adatok mentése/lekérése az adatbázisba/adatbázisból.

# 9. Hitelesítés és jogosultságkezelés

Hitelesítés: JWT alapú token rendszer (src/utils/jwtoken.js). A bejelentkezéskor a backend létrehoz egy aláírt JWT-t, amelyet a kliens tárol (localStorage). A authMiddleware.js ellenőrzi a token érvényességét és a felhasználó szerepkörét.

Fontos szempontok:

- A JWT_SECRET soha ne kerüljön verziókezelésbe.
- Token élettartamának mérlegelése: rövidebb élettartam, célszerű 1 óránál kevesebbet adni.
- Jelszavakat bcrypt-tel kell hash-elni felhasználó biztonság érdekében.

# 10. Fájlfeltöltés és statikus fájlok

Feltöltések (profilkép, termék képek) kezelése a multer technológia segítségével. Feltöltött fájlok tárolhatók a szerveren az uploads/ mappán belül a pfPicture mappában vagy a termekek mappában vagy külső tárhelyen (pl. S3).

Biztonsági javaslatok:

- Ellenőrizni a fájl típusát (MIME) és kiterjesztést.
- Korlátozni a fájlméretet.
- Ne tároljunk végrehajtható fájlokat közkönyvtárban.
- Ha lehetséges, generáljunk egyedi fájlneveket és mappastruktúrát.
- Vissza keresni régebbi, nem használatban lévő képeket, amelyeket ki lehet törölni.

10. Adatbázis indexelés és teljesítmény

Indexek: minden keresési mezőre, külső kulcsokra és gyakran használt WHERE feltételekre indexet kell létrehozni. Példák:

- User.email (UNIQUE index)
- Rendelesek.user_id
- RendelesTermekek.rendeles_id, termek_id

Kapcsolódó teljesítmény-javaslatok:

- N+1 lekérdezések elkerülése JOIN vagy eager-loading használatával.
- Oldalazás (pagination) nagy listázásoknál.
- Cache réteg alkalmazása (pl. Redis) ha szükséges gyakori lekérésekhez.

11. Tranzakciók és adatkonzisztencia

Több táblát érintő műveleteknél (pl. rendelés létrehozása + készlet csökkentése) használjunk tranzakciókat, hogy elkerüljük az inkonzisztens állapotot. Az ORM (Sequelize) tranzakció-támogatást nyújt, de natív driverrel is kezelhető.

# 11. Hibakezelés és naplózás

Hibakezelés:

- Minden aszinkron folyamatban legyen központi hibakezelő (Express error handler).
- Tisztességes HTTP státuszkódok és érthető hibaüzenetek kifelé; belső részletek ne kerüljenek ki production környezetben.

Naplózás:

- Fejlesztésben console-t és Trello-t használjunk.
- Naplózzuk a hibákat, figyelmeztetéseket és fontos üzleti eseményeket (pl. rendelést sikeresen leadta egy felhasználó).

# 12. Biztonsági ajánlások

- Használjunk HTTPS-t minden környezetben (TLS termináció).
- Szerepkör alapú hozzáférés-ellenőrzés (RBAC) az admin és felhasználói műveletekhez.
- Input validáció és output sanitizálás (SQL injection, XSS ellen).
- Limitáljuk az API hívásokat rate limitinggel (pl. express-rate-limit).
- Rendszeres dependency audit (npm audit) és frissítések.

## 13. Backup és restore stratégia

- Rendszeres mentések (napi vagy óránkénti, üzleti igények szerint).
- Mentéseket tároljuk külön zónában, verziózva és titkosítva.
- Időnként teszteljük a visszaállítási folyamatot.
- Minden nagyobb változás előtt mentsük az előző program minden részét egy helyi driverre, ha az új verzió meghibásodna, vagy teljesen össze omlik legyen egy biztonságos, működő verzió.

## 14. Tesztelés

- Unit tesztek kontroller, util és model rétegekhez (pl. Jest, Mocha).
- Integrációs tesztek API végpontokra (supertest).
- End-to-end tesztek a frontend-backend integráció ellenőrzésére.
- Nagy Bum teszt a teljes felhasználói élmény ellenőrzésére és a teljes szerver működésének tesztelésére.
- Fekete dobozos teszt az elvárt státusz kódok és kimeneti adatok tesztelésére.
- Fehér dobozos teszt egy normális, informatikát egyáltalán nem ismerő tesztelő segítségével a felhasználói élmény és az oldal érthetőségének tesztelésére.

## 15. Telepítés és CI

- CI: linters, unit tesztek, integrációs tesztek futtatása PR pipeline-ban.
- Környezetek: dev, staging, production. Minden környezethez külön adatbázis és konfiguráció.

# 16. API végpontok részletes lista (példa)

A következő táblázat részletesen összefoglalja a rendszer főbb API végpontjait. A URL oszlop példaként a fejlesztői környezetben használatos formátumban (localhost:3000) mutatja a címet, a Bemeneti adatok oszlop felsorolja a kéréshez szükséges mezőket és típusukat, a Státuszkód a sikeres válasz HTTP kódját tartalmazza, a Leírás pedig röviden elmagyarázza a végpont célját.

# 17. API Végpontok Teljes Lista

## Felhasználó (User) Végpontok - /api/auth

| Művelet | URL | Bemeneti adatok | Sikeres státuszkód | Leírás | Hibás válaszok |
|---|---|---|---|---|---|
| Regisztráció | http://localhost:3000/api/auth/register | nev, email, jelszo, jelszo_megint, telefonszam | 201 | Létrehoz egy új felhasználói fiókot | 400 (hiányzó mezők), 409 (már létezik email) |
| Bejelentkezés | http://localhost:3000/api/auth/login | email, jelszo | 200 | Bejelentkezteti a felhasználót, JWT token-t ad vissza | 400 (hiányzó mezők), 401 (hibás email vagy jelszó) |
| Kijelentkezés | http://localhost:3000/api/auth/logout | (Authorization fejléc) | 200 | Kijelentkezteti a felhasználót | 401 (nincs token) |
| Profil lekérése | http://localhost:3000/api/auth/profile | (Authorization fejléc) | 200 | Lekéri a bejelentkezett felhasználó adatait | 401 (nincs token), 500 (szerver hiba) |
| Profil módosítása | http://localhost:3000/api/auth/modifyUser | nev, telefonszam, currentPassword, newPassword, profileImage (opcionális) | 200 | Módosítja a felhasználó adatait | 400 (nem egyeznek a jelszavak), 401 (hibás jelszó), 500 (szerver hiba) |
| Borbélyok lekérése | http://localhost:3000/api/auth/borbelyok | - | 200 | Lekéri az összes borbélyt | 400 (nincsenek borbélyok), 500 (szerver hiba) |
| Borbélynevek | http://localhost:3000/api/auth/borbelyokNeve | - | 200 | Lekéri a borbélyok neveit | 400 (nincsenek borbélyok), 500 (szerver hiba) |
| Felhasználó törlése | http://localhost:3000/api/auth/deleteUser | (Authorization fejléc) | 200 | Törli a bejelentkezett felhasználót | 400 (felhasználó nem található), 500 (szerver hiba) |

## Admin Felhasználó Végpontok - /api/auth/admin

| Művelet | URL | Bemeneti adatok | Sikeres státuszkód | Leírás | Hibás válaszok |
|---|---|---|---|---|---|

| | | | | | |
|---|---|---|---|---|---|
| Admin bejelentkezés | http://localhost:3000/api/auth/admin/login | email, jelszo | 200 | Admin bejelentkeztetés, JWT token-t ad vissza | 400 (hiányzó mezők), 401 (hibás email vagy jelszó), 500 (szerver hiba) |
| Felhasználó létrehozása | http://localhost:3000/api/auth/admin/createUser | nev, email, jelszo, jelszo_megint, role, telefonszam | 200 | Admin létrehoz egy új felhasználót | 400 (hiányzó mezők, nem egyeznek a jelszavak), 500 (szerver hiba) |
| Felhasználó módosítása | http://localhost:3000/api/auth/admin/modifyUser | id, nev, email, jelszo, telefon, role | 200 | Admin módosít egy felhasználót | 400 (hiányzó mezők), 401 (nem létező szerepkör), 500 (szerver hiba) |
| Felhasználó törlése | http://localhost:3000/api/auth/admin/deleteUser | id | 200 | Admin töröl egy felhasználót | 400 (felhasználó nem található), 500 (szerver hiba) |
| Felhasználó keresése név alapján | http://localhost:3000/api/auth/admin/getUserByName | nev | 200 | Keres felhasználót név alapján | 400 (nincs kitöltve a mező), 500 (szerver hiba) |
| Felhasználó keresése email alapján | http://localhost:3000/api/auth/admin/getUserByEmail | email | 200 | Keres felhasználót email alapján | 400 (nincs kitöltve a mező), 500 (szerver hiba) |

## Termék (Termek) Végpontok - /api/termek

| Művelet | URL | Bemeneti adatok | Sikeres státuszkód | Leírás | Hibás válaszok |
|---|---|---|---|---|---|
| Terméklista | http://localhost:3000/api/termek/products | name (opcionális), category (opcionális) | 200 | Lekéri a termékeket, szűrési lehetőséggel | 500 (szerver hiba) |

## Admin Termék Végpontok - /api/termek/admin

| Művelet | URL | Bemeneti adatok | Sikeres státuszkód | Leírás | Hibás válaszok |
|---|---|---|---|---|---|
| Termék létrehozása | http://localhost:3000/api/termek/admin/productsCreate | file, name, category, price, comment | 200 | Létrehoz egy új terméket | 400 (hiányzó kép), 401 (hiányzó mezők), 500 (szerver hiba) |
| Termék módosítása | http://localhost:3000/api/termek/admin/productsModify | id, file (opcionális), name, price, comment | 200 | Módosít egy terméket | 400 (termék nem található), 404 (termék nem található az adatbázisban), 500 (szerver hiba) |

| | | | | | |
|---|---|---|---|---|---|
| Termék törlése | http://localhost:3000/api/termek/admin/productDelete | id | 200 | Töröl egy terméket | 400 (termék nem található), 500 (szerver hiba) |
| Termék keresése név alapján | http://localhost:3000/api/termek/admin/getProductByName | nev | 200 | Keres terméket név alapján | 401 (nincs kitöltve a mező), 404 (termék nem található), 500 (szerver hiba) |
| Termék keresése kategória alapján | http://localhost:3000/api/termek/admin/getProductsByCategory | category | 200 | Keres termékeket kategória alapján | 400 (nincs kitöltve a mező), 500 (szerver hiba) |

## Rendelés (Rendeles) Végpontok - /api/rendeles

| Művelet | URL | Bemeneti adatok | Sikeres státuszkód | Leírás | Hibás válaszok |
|---|---|---|---|---|---|
| Rendelés létrehozása | http://localhost:3000/api/rendeles/rendelesCreate | termekek, iranyitoszam, telepules, szallitasiCim, ar | 201 | Létrehoz egy új rendelést | 400 (üres kosár, hiányzó címek), 404 (felhasználó nem található), 500 (szerver hiba) |
| Rendelés törlése | http://localhost:3000/api/rendeles/rendelesDelete | id | 200 | Töröl egy rendelést | 400 (rendelés nem található), 500 (szerver hiba) |
| Rendelés lekérése | http://localhost:3000/api/rendeles/getRendelesById | id | 200 | Lekéri a rendelés adatait | 400 (nincs ilyen rendelés), 401 (nincs megadva a kód), 500 (szerver hiba) |

## Admin Rendelés Végpontok - /api/rendeles/admin

| Művelet | URL | Bemeneti adatok | Sikeres státuszkód | Leírás | Hibás válaszok |
|---|---|---|---|---|---|
| Rendelés módosítása | http://localhost:3000/api/rendeles/admin/modifyRendeles | id, vasarloNeve, vasarloEmail, telefonszam, iranyitoszam, telepules, szallitasiCim, rendelesIdeje, termekek | 200 | Módosít egy rendelést | 500 (szerver hiba) |
| Rendelés törlése | http://localhost:3000/api/rendeles/admin/deleteRendeles | id | 200 | Töröl egy rendelést | 400 (rendelés nem található), 500 (szerver hiba) |
| Rendelés keresése név alapján | http://localhost:3000/api/rendeles/admin/getRendelesByName | nev | 200 | Keres rendelést név alapján | 400 (nincs kitöltve a mező, rendelés nem található), 500 (szerver hiba) |

| | | | | | |
|---|---|---|---|---|---|
| Rendelés keresése email alapján | http://localhost:3000/api/rendeles/admin/getRendelesByEmail | email | 200 | Keres rendelést email alapján | 500 (szerver hiba) |

## Foglalás (Booking) Végpontok - /api/foglalas

| Művelet | URL | Bemeneti adatok | Sikeres státuszkód | Leírás | Hibás válaszok |
|---|---|---|---|---|---|
| Összes foglalás | http://localhost:3000/api/foglalas/getEveryFoglalas | - | 200 | Lekéri az összes foglalást | 500 (szerver hiba) |
| Foglalás létrehozása | http://localhost:3000/api/foglalas/foglalasCreate | nev, email, telefonszam, borbely, idopont, szolgaltatas, ar, userId, borbelyId | 200 | Létrehoz egy új foglalást | 400 (hiányzó mezők), 500 (szerver hiba) |
| Foglalás lekérése | http://localhost:3000/api/foglalas/getFoglalasById | id | 200 | Lekéri a foglalás adatait | 400 (nincs ilyen foglalás), 401 (nincs megadva a kód), 500 (szerver hiba) |
| Foglalás módosítása | http://localhost:3000/api/foglalas/foglalasModify | id, és egyéb módosítandó mezők | 200 | Módosít egy foglalást | 400 (foglalás nem található, nincs módosítandó adat), 500 (szerver hiba) |
| Foglalás törlése | http://localhost:3000/api/foglalas/foglalasDelete | id | 200 | Töröl egy foglalást | 400 (foglalás nem található, nincs ilyen kódú foglalás), 500 (szerver hiba) |

## Admin Foglalás Végpontok - /api/foglalas/admin

| Művelet | URL | Bemeneti adatok | Sikeres státuszkód | Leírás | Hibás válaszok |
|---|---|---|---|---|---|
| Foglalás módosítása | http://localhost:3000/api/foglalas/admin/modifyFoglalas | id, és egyéb módosítandó mezők | 200 | Admin módosít egy foglalást | 400 (foglalás nem található, nincs módosítandó adat), 500 (szerver hiba) |
| Foglalás törlése | http://localhost:3000/api/foglalas/admin/foglalasDelete/ | id | 200 | Admin töröl egy foglalást | 400 (foglalás nem található), 500 (szerver hiba) |
| Foglalás keresése név alapján | http://localhost:3000/api/foglalas/admin/getFoglalasByName | nev | 200 | Keres foglalást név alapján | 400 (nincs kitöltve a mező), 500 (szerver hiba) |
| Foglalás keresése email alapján | http://localhost:3000/api/foglalas/admin/getFoglalasByEmail | email | 200 | Keres foglalást email alapján | 400 (nincs kitöltve a mező), 500 (szerver hiba) |

## Feltöltés (Upload) Végpontok

| Művelet | URL | Bemeneti adatok | Sikeres státuszkód | Leírás | Hibás válaszok |
|---|---|---|---|---|---|
| Termék kép lekérése | http://localhost:3000/api/uploads/termekek/:id | id (útvonalparaméter) | 200 | Lekéri a termék kép adatait | 400 (érvénytelen termék kép ID), 500 (szerver hiba) |

## Általános Hibaüzenetek

| Státuszkód | Jelentés | Gyakori ok |
|---|---|---|
| 400 | Bad Request | Hiányzó vagy hibás bemeneti adatok |
| 401 | Unauthorized | Hiányzó vagy érvénytelen token, hitelesítési hiba |
| 403 | Forbidden | Nincs jogosultság az adott művelethez |
| 404 | Not Found | A keresett erőforrás nem található |
| 409 | Conflict | Az erőforrás már létezik (pl. email) |
| 500 | Internal Server Error | Szerver oldali hiba |

## Megjegyzések

- Az összes `/api/auth`, `/api/termek/admin`, `/api/rendeles/admin`, `/api/foglalas/admin` végpontokhoz Authorization header szükséges (Bearer token)
- Az admin végpontokhoz additionally `isAdmin` middleware ellenőrzés szükséges
- A feltöltéshez multipart/form-data formátum szükséges

# 18. Adat szótár (példák)

## 18.1 Projekt Technikai Szójegyzéke

Ez a dokumentum a Barbershop projektben használt idegen és szakmai kifejezéseket tartalmazza részletes magyar magyarázatokkal.

*1. Szerepkörök és jogosultságok (Role)*

**Admin (Rendszergazda)**

- **Jelentés:** Olyan felhasználó, akinek teljes körű joga van a rendszerhez.

- **Jogosultságok:**
    - Felhasználók létrehozása, módosítása és törlése
    - Termékek kezelése (létrehozás, módosítás, törlés)
    - Foglalások kezelése és törlése
    - Rendelések kezelése és törlése
    - Más felhasználók adatainak megtekintése név vagy email alapján
- **Hozzáférés:** Az adminisztrátori funkciók csak a /admin útvonalon érhetők el, és minden művelethez isAdmin middleware ellenőrzés szükséges.

**Fogyasztó (User/Felhasználó)**

- **Jelentés:** A rendszer alapvető felhasználói csoportja.
- **Jogosultságok:**
    - Regisztráció és bejelentkezés
    - Profil adatok megtekintése és módosítása
    - Foglalás időpontokra
    - Rendelések leadása és saját rendeléseinek megtekintése
    - Termékek böngészése és kosár használata

**Borbély (Barber)**

- **Jelentés:** Speciális felhasználó, aki szolgáltatásokat nyújt a fogyasztóknak.
- **Jogosultságok:**
    - Bejelentkezés a rendszerbe
    - Naptár megtekintése (milyen foglalások vannak rá)
    - A saját foglalásainak kezelése

*2. Hitelesítés és biztonság (Authentication & Security)*

**Password (Jelszó)**

- **Jelentés:** A felhasználó azonosítására szolgáló titkos karaktersorozat.
- **Tárolás:** A jelszó soha nemplaintext (szöveges) formában tárolódik. A projektben a **bcrypt** algoritmus használatával történik a hash-elés (titkosítás). A bcrypt egy

egyirányú titkosítási algoritmus, ami azt jelenti, hogy a tárolt hash-ből nem lehet visszaállítani az eredeti jelszót.

- **Példa:** bcrypt.hash(password, 12) - a 12 a költségfaktor (cost factor), ami meghatározza a hash-elés bonyolultságát.

**JWT (JSON Web Token)**

- **Jelentés:** A hitelesítés alapjául szolgáló token, amely a felhasználó adatait tartalmazza JSON formátumban, digitálisan aláírva.
- **Működés:**
  i. A felhasználó bejelentkezéskor kap egy JWT tokent a szervertől
  ii. Ezt a tokent a kliens (frontend) eltárolja (localStorage)
  iii. Minden kérésnél a token elküldésre kerül az Authorization header-ben
  iv. A szerver ellenőrzi a token érvényességét
- **Tartalom:** A token tartalmazza a felhasználó id, email és role (szerepkör) adatait.
- **Példa:** Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

**Token**

- **Jelentés:** Általános értelemben egy azonosító, amely a hitelesítési folyamat során keletkezik.
- **Típus:** Ebben a projektben JWT tokenről van szó.
- **Élettartam:** A token JWT_EXPIRES környezeti változóban meghatározott ideig érvényes (pl. 7d = 7 nap).

**Auth Middleware (Hitelesítési köztesréteg)**

- **Jelentés:** Egy szoftverréteg, amely minden védett kérés előtt lefut és ellenőrzi, hogy a felhasználó be van-e jelentkezve érvényes tokennel.
- **Funkció:**
  - Ellenőrzi, hogy a kérés tartalmaz-e token
  - Validálja a token formátumát (Bearer schema)
  - Dekódolja és ellenőrzi a token aláírását
  - Ha érvénytelen vagy hiányzik a token, 401-es HTTP hibakódot ad vissza
- **Fájl:** src/middlewares/authMiddleware.js

**isAdmin Middleware**

- **Jelentés:** Speciális middleware, amely ellenőrzi, hogy a bejelentkezett felhasználó Admin szerepkörű-e.
- **Funkció:** Csak az Admin szerepkörű felhasználók férhetnek hozzá az admin funkciókhoz.
- **Példa használat:** route.post('/admin/createUser', authMiddleware, isAdmin, userController.adminCreateUser)

**bcrypt**

- **Jelentés:** Egy jelszó hash-elésére használt Node.js könyvtár.
- **Funkció:** A jelszavakat biztonságos, visszafejthetetlen formában tárolja.
- **Példa:** await bcrypt.hash(password, 12) - hash-elés; await bcrypt.compare(password, hash) - jelszó ellenőrzés.

*3. Adatbázis és ORM (Database & ORM)*

**Sequelize**

- **Jelentés:** Egy ORM (Object-Relational Mapping) könyvtár Node.js-hez, amely lehetővé teszi az adatbázis műveletek programozási nyelven történő végrehajtását SQL helyett.
- **Funkció:**
  - Adatbázis kapcsolat kezelése
  - Modellek (adattáblák) definiálása
  - CRUD műveletek végrehajtása
  - Kapcsolatok (relációk) kezelése a táblák között
- **Használat:** A projektben MariaDB/MySQL adatbázissal használják.

**Model (Adatmodell)**

- **Jelentés:** Az adatbázis táblájának JavaScript/TypeScript osztályként való reprezentációja.
- **Példák a projektben:**
  - userModel.js - felhasználók táblája
  - termekModel.js - termékek táblája

    - rendelesekModel.js - rendelések táblája
    - foglalasModel.js - foglalások táblája

**Migration ( Migráció)**

- **Jelentés:** Az adatbázis séma (szerkezet) változtatásainak verziókövetett leírása.
- **Cél:** A projektben a db.sync() automatikusan kezeli a táblák létrehozását.

**Seeder (Seedelő)**

- **Jelentés:** Olyan script, amely kezdeti (demo) adatokat tölt fel az adatbázisba.
- **Példák a projektben:**
    - adminSeeder.js - létrehozza az alapértelmezett admin felhasználót
    - termekSeeder.js - feltölti a kezdeti termékeket

**Foreign Key (Külső kulcs)**

- **Jelentés:** Egy táblában lévő mező, amely egy másik táblára mutat, ezzel kapcsolatot teremtve közöttük.
- **Példa:** A rendelesek táblában a user_id mező a users táblára mutat.

**Primary Key (Elsődleges kulcs)**

- **Jelentés:** Egyedi azonosító minden rekordhoz a táblában.
- **Példa:** A legtöbb táblában az id mező az elsődleges kulcs, amely auto-increment (automatikusan növekvő) típusú.

**Relation (Kapcsolat/Reláció)**

- **Jelentés:** Az adatbázis táblák közötti kapcsolat leírása.
- **Típusok:**
    - One-to-One (1:1) - egy felhasználóhoz egy profil
    - One-to-Many (1:N) - egy felhasználónak több rendelése lehet
    - Many-to-Many (N:N) - egy rendelés több terméket tartalmaz, egy termék több rendelésben szerepelhet

*4. Backend architektúra (Backend Architecture)*

**Controller (Kontroller)**

- **Jelentés:** A backend logikáját tartalmazó fájl, amely kezeli a kéréseket és válaszokat.
- **Funkció:**
    - Beérkező adatok validálása
    - Adatbázis műveletek végrehajtása (modellek használatával)
    - Válasz küldése a kliensnek
- **Példák:** userController.js, termekController.js, rendelesekController.js, foglalasController.js

**Route (Útvonal/Végpont)**

- **Jelentés:** Az API elérési útja, amely meghatározza, hogy egy adott URL és HTTP metódus esetén melyik kontroller függvény fusson le.
- **Példák:**
    - POST /api/auth/register - regisztráció
    - POST /api/auth/login - bejelentkezés
    - GET /api/termek/products - termékek listázása

**Endpoint (Végpont)**

- **Jelentés:** Ugyanaz, mint a Route - egy API elérési pont.
- **Példa:** "A /api/auth/login endpoint-on keresztül lehet bejelentkezni."

**Middleware (Köztesréteg)**

- **Jelentés:** Olyan függvény, amely a kérés és a válasz között fut le.
- **Típusok a projektben:**
    - authMiddleware.js - hitelesítés ellenőrzése
    - uploads.js - fájlfeltöltés kezelése

**API (Application Programming Interface)**

- **Jelentés:** Az alkalmazások közötti kommunikációt lehetővé tevő interfész.
- **REST API:** A projektben RESTful API stílust használnak, ahol:
    - GET - adat lekérése
    - POST - új adat létrehozása
    - PUT - adat módosítása

- DELETE - adat törlése

**Server (Szerver)**

- **Jelentés:** A backend alkalmazást futtató szerver.
- **Fájl:** server.js
- **Port:** Alapértelmezetten 3000 (konfigurálható a PORT környezeti változóval)

*5. Frontend architektúra (Frontend Architecture)*

**Component (Komponens)**

- **Jelentés:** Az Angular alkalmazás önálló, újrafelhasználható része.
- **Példák:** home, login, register, profil, termekek, foglalas, admin, kosar

**Service (Szolgáltatás)**

- **Jelentés:** Olyan osztály, amely az üzleti logikát és az adatok kezelését végzi, jellemzően HTTP kéréseket intéz a backend felé.
- **Példa:** kosar.services.ts - a kosár kezeléséhez szükséges függvények

**HTTP**

- **Jelentés:** A frontend és backend közötti kommunikáció protocollja.
- **Angular használat:** HttpClient modulon keresztül történik a backend API hívása.

**LocalStorage**

- **Jelentés:** A böngészőben tárolt kulcs-érték párok, amelyek a munkamenet után is megmaradnak.
- **Használat:** A projektben a token és a user (bejelentkezett felhasználó adatai) eltárolása.

**Routing (Útvonal-vezérlés)**

- **Jelentés:** A különböző oldalak (komponensek) közötti navigáció kezelése.
- **Fájl:** app-routing-module.ts

**Module (Modul)**

- **Jelentés:** Az Angular alkalmazás logikai egysége, amely összefogja a komponenseket, szolgáltatásokat és más elemeket.
- **Fájl:** app-module.ts

*6. Fájlfeltöltés (File Upload)*

**Multer**

- **Jelentés:** Node.js middleware a multipart/form-data kezelésére, azaz fájlfeltöltésre.
- **Funkció:** Képek (termék képek, profil képek) feltöltésének kezelése.

**Uploads (Feltöltések)**

- **Jelentés:** A feltöltött fájlok tárolására szolgáló mappa.
- **Almappák:**
    - uploads/pfpicture/ - profilképek
    - uploads/termekek/ - termékképek

**Profile Image / PfPicture (Profilkép)**

- **Jelentés:** A felhasználó által beállított kép, amely a profilját reprezentálja.
- **Tárolás:** A feltöltött képek az uploads/pfpicture/ mappában, fájlnévvel (pl. UUID vagy timestamp alapján).

*7. HTTP státuszkódok (HTTP Status Codes)*

**200 OK**

- **Jelentés:** A kérés sikeresen lefutott.

**201 Created**

- **Jelentés:** Új erőforrás (pl. felhasználó, termék) sikeresen létrehozva.

**400 Bad Request**

- **Jelentés:** A kérés hibás vagy hiányos adatokat tartalmaz.

**401 Unauthorized**

- **Jelentés:** Nincs hitelesítés (nincs token vagy érvénytelen token).

**403 Forbidden**

- **Jelentés:** Nincs jogosultság a művelethez (pl. nem Admin).

**500 Internal Server Error**

- **Jelentés:** Szerver oldali hiba.

*8. Egyéb technikai kifejezések*

**CRUD**

- **Jelentés:** Create (Létrehozás), Read (Olvasás), Update (Módosítás), Delete (Törlés) - az alapvető adatkezelő műveletek.

**Environment Variables (Környezeti változók)**

- **Jelentés:** A rendszer konfigurációs beállításai, amelyek a .env fájlban tárolódnak és nem kerülnek verziókezelésbe.
- **Példák:**
  - DB_HOST - adatbázis szerver címe
  - DB_USER - adatbázis felhasználó
  - DB_PASS - adatbázis jelszó
  - JWT_SECRET - JWT aláírás titkos kulcsa

**.env fájl**

- **Jelentés:** Konfigurációs fájl, amely tartalmazza a környezeti változókat.
- **Fontos:** Soha ne kerüljön verziókezelésbe (Git), csak a .env.example!

**Package.json**

- **Jelentés:** A Node.js projekt fájlja, amely tartalmazza a függőségeket és scripteket.

**npm install**

- **Jelentés:** Parancs a projekt függőségeinek telepítésére a package.json alapján.

**Express**

- **Jelentés:** Node.js keretrendszer webalkalmazások és API-k készítéséhez.

**Node.js**

- **Jelentés:** JavaScript futtatási környezet szerveroldalon.

**Angular**

- **Jelentés:** Google által fejlesztett TypeScript keretrendszer frontend alkalmazásokhoz.

**TypeScript**

- **Jelentés:** A JavaScript típusokkal kibővített változata.

**CORS (Cross-Origin Resource Sharing)**

- **Jelentés:** Biztonsági mechanizmus, amely engedélyezi vagy blokkolja a különböző domain-ek közötti kéréseket.

**Database (Adatbázis)**

- **Jelentés:** Strukturált adatok tárolására szolgáló rendszer.
- **Használt adatbázis:** MariaDB vagy MySQL

**SQL (Structured Query Language)**

- **Jelentés:** Adatbázis lekérdező és kezelő nyelv.

*9. Üzleti entitások (Business Entities)*

**User / Felhasználó**

- **Mezők:** id, nev (név), jelszo (jelszó hash), email, telefonszam, pfPicture (profil kép), foglaltsag (szerepkör)

**Termek / Termék**

- **Mezők:** id, name (név), description (leírás), price (ár), stock (készlet), image (kép)

**Rendeles / Rendelés**

- **Mezők:** id, user_id (felhasználó), total_price (összes ár), status (állapot), created_at

**Foglalas / Foglalás**

- **Mezők:** id, user_id (felhasználó), borbely_id (borbély), idopont (időpont), szolgaltatas (szolgáltatás), status (állapot)

**RendelesTermekek / Rendelés-termék kapcsolat**

- **Mezők:** id, rendeles_id (rendelés), termek_id (termék), quantity (mennyiség), unit_price (egységár)

*10. Szolgáltatások (Services - a foglalas komponensben)*

**Hajvágás**

- **Ár:** 6490 Ft
- **Időtartam:** 45 perc

**Szakállvágás**

- **Ár:** 3990 Ft
- **Időtartam:** 30 perc

**Hajvágás + Szakáll**

- **Ár:** 8990 Ft
- **Időtartam:** 60 perc

*11. Státuszok (Status)*

**User foglaltsag (szerepkör)**

- Fogyasztó - normál felhasználó
- Borbély - fodrász szolgáltató
- Admin - rendszergazda

**Rendeles status (állapot)**

- A projektben különböző állapotok lehetnek: pending (függőben), paid (fizetve), shipped (kiszállítva), cancelled (lemon dva)

**Foglalas status (állapot)**

- booked (lefoglalva)
- completed (befejezve)
- cancelled (lemondva)

*12. Email és levelezés*

**SMTP (Simple Mail Transfer Protocol)**

- **Jelentés:** Levelek küldésének protocollja.
- **Konfiguráció:** A config/mail.js fájlban található a mail küldés beállítása.

**Mail Host/Port/User/Pass**

- **Jelentés:** Az SMTP szerver eléréséhez szükséges adatok (környezeti változókban tárolva).

*13. Egyéb projekt-specifikus kifejezések*

**ADMIN_PASS**

- **Jelentés:** Speciális jelszó, amelynek használatakor a regisztráló felhasználó automatikusan Admin szerepkört kap.
- **Használat:** A regisztrációs űrlapon ha valaki ezt a jelszót adja meg, Admin lesz, ellenkező esetben Fogyasztó.

**Kosar / Shopping Cart**

- **Jelentés:** A termékek ideiglenes tárolása vásárlás előtt.
- **Működés:** A frontend kosar.services.ts service-ben kezelt kosár.

**Borbely-naptar**

- **Jelentés:** A borbélyok naptár nézete, ahol a foglalásaik láthatók.

## 18.2 Összefoglaló táblázat

| **Angol kifejezés** | **Magyar jelentés** | **Rövid magyarázat** |
|---|---|---|
| Admin | Rendszergazda | Teljes körű hozzáférésű felhasználó |
| Password | Jelszó | Felhasználó azonosítója, bcrypt-pel titkosítva |
| JWT | JSON Web Token | Hitelesítési token |
| Token | Token | Azonosító karaktersorozat |
| Auth | Hitelesítés | Bejelentkezés ellenőrzése |

| **Angol kifejezés** | **Magyar jelentés** | **Rövid magyarázat** |
|---|---|---|
| Middleware | Köztesréteg | Kérés előtt futó ellenőrző kód |
| Controller | Kontroller | Backend logika |
| Model | Modell | Adatbázis tábla reprezentáció |
| Route | Útvonal | API elérési pont |
| Endpoint | Végpont | API elérési pont |
| Seeder | Seedelő | Kezdeti adatok feltöltése |
| bcrypt | bcrypt | Jelszó titkosító könyvtár |
| Role | Szerepkör | Felhasználó jogosultsági szintje |
| Email | Email | Elektronikus levélcím |
| Register | Regisztráció | Új felhasználó létrehozása |
| Login | Bejelentkezés | Azonosítás |
| Logout | Kijelentkezés | Session megszüntetése |
| Sequelize | Sequelize | ORM könyvtár |

| **Angol kifejezés** | **Magyar jelentés** | **Rövid magyarázat** |
|---|---|---|
| Migration | Migráció | Adatbázis séma változtatás |
| Foreign Key | Külső kulcs | Táblák közti kapcsolat |
| Primary Key | Elsődleges kulcs | Egyedi azonosító |
| Relation | Kapcsolat | Táblák közötti kapcsolat |
| API | API | Alkalmazás interfész |
| Server | Szerver | Backend alkalmazás futtató |
| Component | Komponens | UI elem |
| Service | Szolgáltatás | Üzleti logika |
| HTTP | HTTP | Protokoll |
| LocalStorage | LocalStorage | Böngésző tárhely |
| Uploads | Feltöltések | Fájltárolás |
| Multer | Multer | Fájlfeltöltő middleware |
| CRUD | CRUD | Alapvető adatműveletek |

| Angol kifejezés | Magyar jelentés | Rövid magyarázat |
|---|---|---|
| .env | .env | Konfigurációs fájl |
| Express | Express | Node.js keretrendszer |
| Angular | Angular | Frontend keretrendszer |
| CORS | CORS | Cross-Origin megosztás |
| Database | Adatbázis | Adattároló |
| Status | Állapot | Művelet/státusz |
| Cart | Kosár | Vásárlás előtti tár |

## 19. Példa lekérdezések és tranzakciós minta (pseudokód)

Rendelés létrehozása (pseudokód):

- Kezd a tranzakciót
- Validáld a kosár tartalmát
- Számold a teljes árat
- Hozd létre a Rendelesek rekordot
- Hozd létre a RendelesTermekek rekordokat
- Commit tranzakció
- Ha hiba van -> rollback és válasz hibával

## 20. Üzemeltetési checklist

- Érvényes TLS tanúsítvány
- Rendszeres dependency frissítések
- Mentések, naplózás, monitoring (pl. Prometheus, Grafana)
- Biztonsági audit (pentest) évente
- Disaster recovery terv és contact lista

## 21. Változtatások és javasolt fejlesztések

- SSO integráció (pl. OAuth2, OpenID Connect) hosszú távú skálázhatóságért
- Külön mikroservicerek az intenzív komponensekhez (pl. rendelések kezelése)
- Async munkamenetek message queue-val (pl. RabbitMQ vagy Kafka) hosszú futású feladatokhoz

## 22. Mellékletek

- ERD diagram: generáld a databaseRelations.erd fájl alapján
- Példakérések Postman vagy OpenAPI specifikáció formájában

## 23. Postman tesztesetek (tervezet)

A következő lépésekben összeállítok néhány Postman gyűjteményt az API igazolására. Az alábbi példák bemutatják, milyen kéréseket és elvárt válaszokat lehet tesztelni. Ezekben a négy alap API kérések szerepelnek: **GET, POST, PUT, DELETE**. Később ezeket a gyűjteményekbe importálva könnyen futtathatod a teljes felhasználói folyamatot. A képekben megadott adatok csak tesztesetek arra kitalálva, hogy a fejlesztőknek és a szoftvert használóknak egyszerűbb legyen az adat típusokat meghatározni és hogy legyen egy konkrét példa a fontosabb API-kra. Mivel JsonWebToken alkalmazunk ezért a legtöbb végponthoz szükség lesz egy token-re. A legegyszerűbben úgy szerzünk token-t, hogy a postman alkalmazásban, vagy a postman Visual Studio Code-ban extension-ként a

http://localhost:3000/api/auth/login végponttal egy POST kérésben lekérünk egy felhasználót (pélául a képben megadott adatokkal) és ott a kapott token-t kimásoljuk és azt beillesztjük a token helyére. Ezek a tokenek 45 perc leteltével lejárnak.

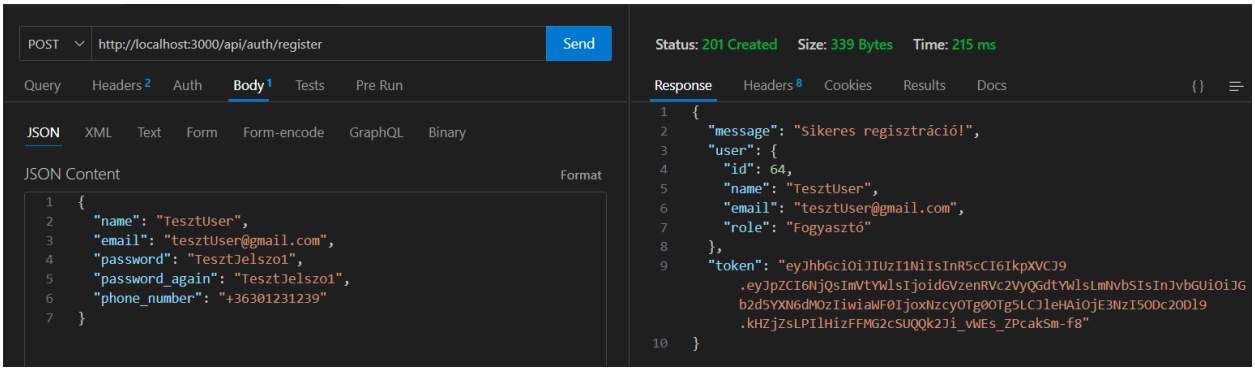
## 1. Felhasználó regisztráció

**POST** /api/auth/register

body

```
{
  "name": "TesztUser",
  "email": "tesztUser@gmail.com",
  "password": "TesztJelszo1",
  "password_again": "TesztJelszo1",
  "phone_number": "+36301231239"
}
```

Várható válasz 201 Created, visszaküldött objektum tartalmazza az id-t és a nev-et az email-t a foglaltságot, de nem a jelszó hasht.



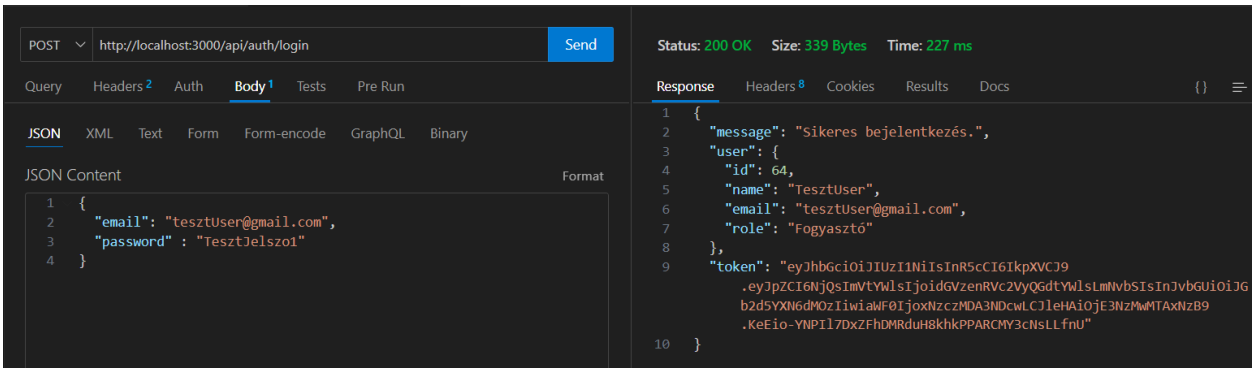

## 2. Bejelentkezés

**POST** /api/auth/login

body

```
{
  "email": "teszt@example.com",
  "password": "Jelszo123!"
}
```

Siker esetén 200 OK jön, a body tartalmazza a token mezőt ({ "token": "...JWT..." }). Ezt a tokent kell a további kéréseknél az Authorization: Bearer <token> fejléccel küldeni és más token-re szükséges API-knál használni.



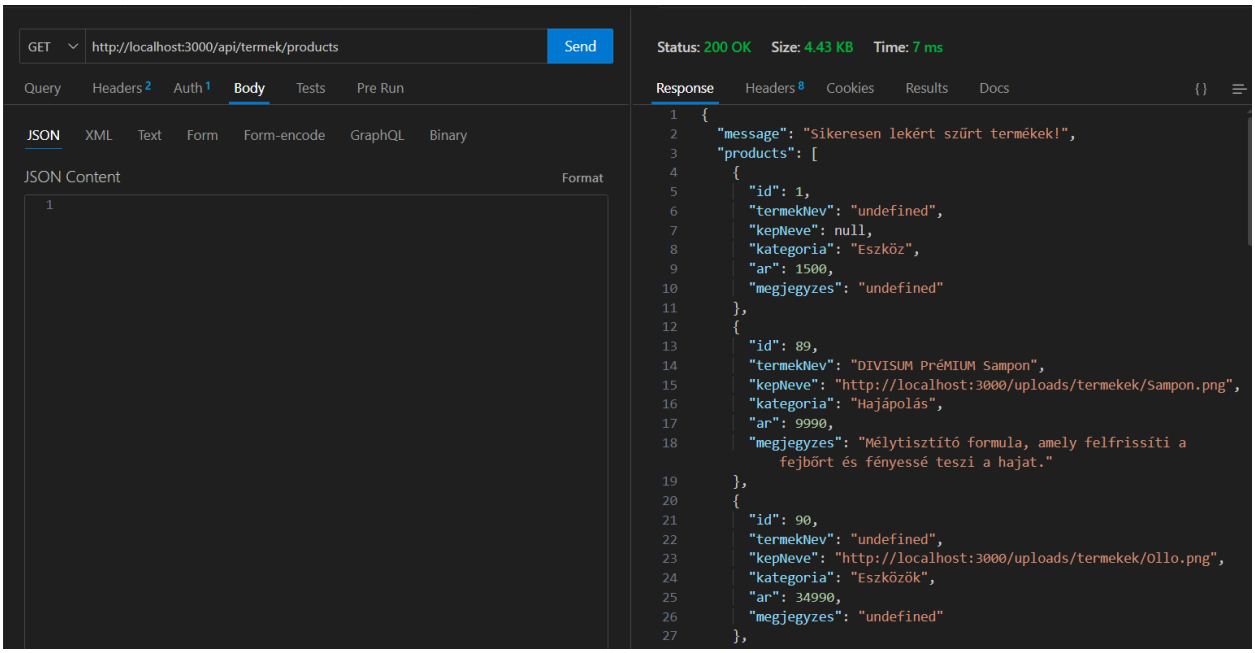

## 3. Terméklista lekérdezése

**GET** /api/termekek/products
Fejlécek:

Authorization: Bearer <token>

Várható 200 OK, a válasz egy tömb, pl.:

[ { "id":1, "termekNev":"Sampon", "ar":499, "kepNeve":http://localhost:3000/uploads/termekek/Sampon.png, "kategoria": "Hajápolás", "megjegyzes": "Mélytisztító…" }, { "id":2, "termekNev":"Olló", "ar":800, "kepNeve":http://localhost:3000/uploads/termekek/Ollo.png, "kategoria": "Eszköz", "megjegyzes": "undefined" } ]



## 4. Rendelés lekérése

**POST** /api/rendeles/getRendelesById

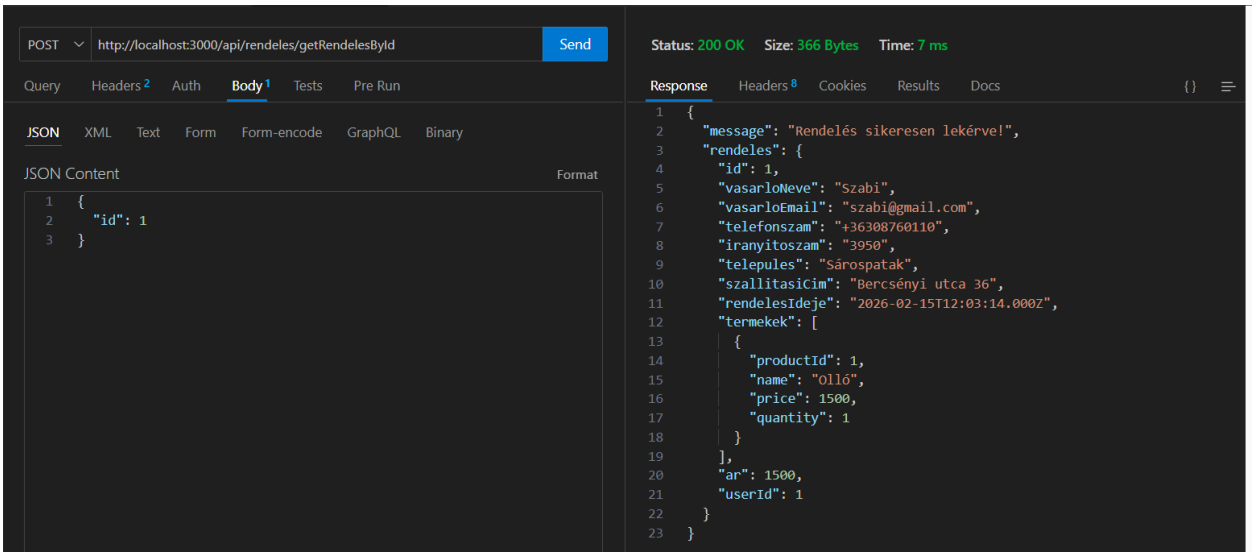
body

{

"id": 1

}

Várható válasz 200 OK, a Response tartalmazza a teljes rendelés id-t, vasarló nevét, email-jét, telefonszámot, irányítószámot, települést, szállítási címet, rendelés idejét, termékeket JSON objektumban: { productid, name, price, quantity }, teljes árat és a felhasználó id-t.

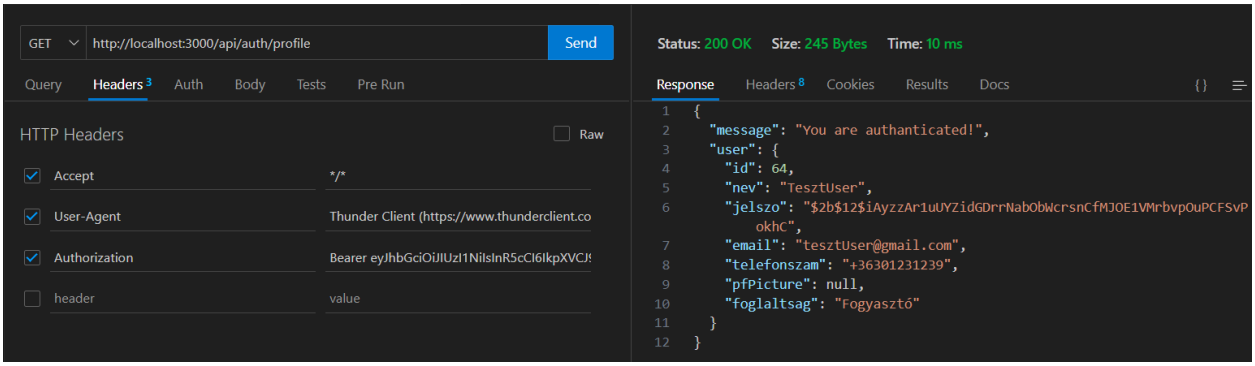



## 5. Profil lekérése

**GET** /api/auth/profile

Fejlécek:

Authorization: Bearer <token>
Várható 200 OK, válasz: a token által lekért Response-ban a felhasználó tartalmazza id-t, nevet, jelszavat, email-t, telefonszámot, profilképet és a foglaltságot.



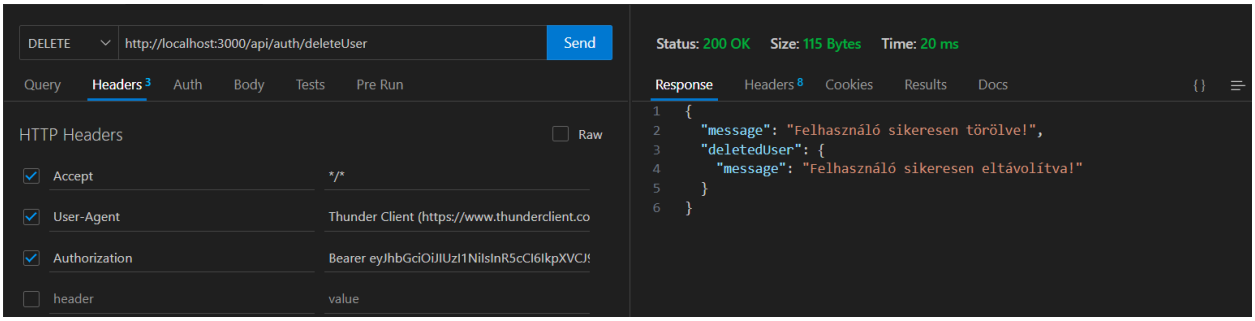

# 6.Profil törlése

**DELETE** /api/auth/deleteUser

Fejlécek

Authorization: Bearer <token>

Várható 200 OK, válasz: a token által elért fiók törlése.




# 7.Profil módosítás

**PUT** /api/auth/modifyUser

Fejlécek

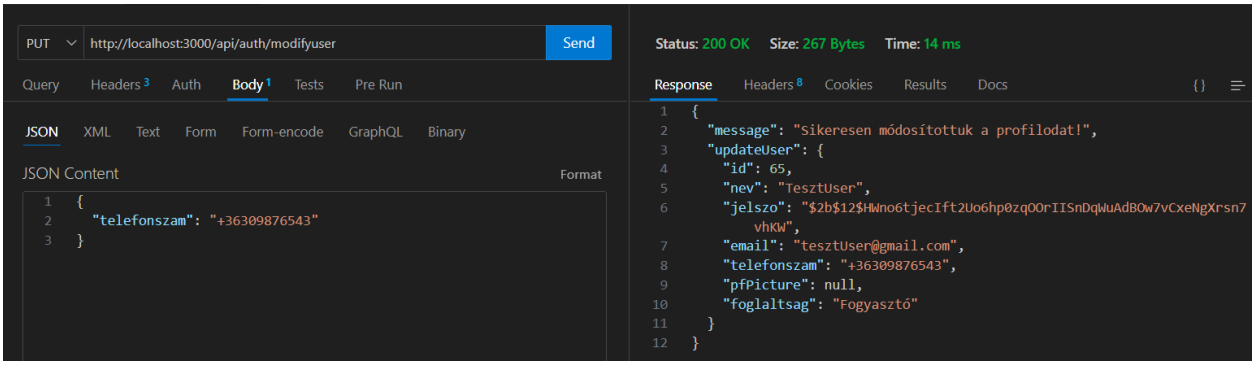
Authorization: Bearer <token>

Body

Amilyen adatot szeretnél változtatni például:

"telefonszam" = " +36308760110"

Várható 200 OK, válasz: a token által elért felhasználó valamennyi adata megváltozik.




# 24. Részletes telepítési útmutató (Backend és Adatbázis)

Ebben a fejezetben lépésről lépésre megadom, hogyan telepítsd és konfiguráld a Backendet és az adatbázist fejlesztési és termelési környezetre, Windows esetén.

## Általános előfeltételek

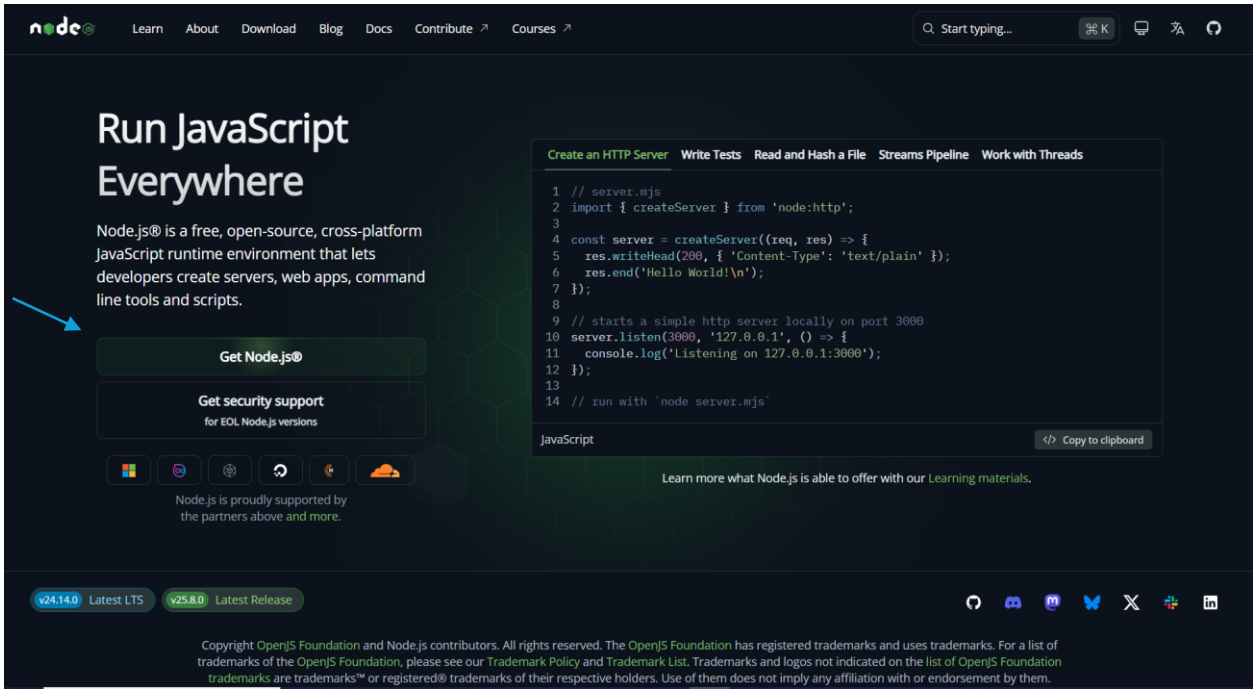
- Node.js (LTS ajánlott, pl. 18.x vagy 20.x).
- NPM.
- MariaDB vagy MySQL szerver (helyi vagy távoli).
- Git (forráskezeléshez).
- Ha Docker-t használsz: Docker Engine és Docker Compose.

## 25.1 Lokális fejlesztés — Windows (PowerShell) lépésről lépésre

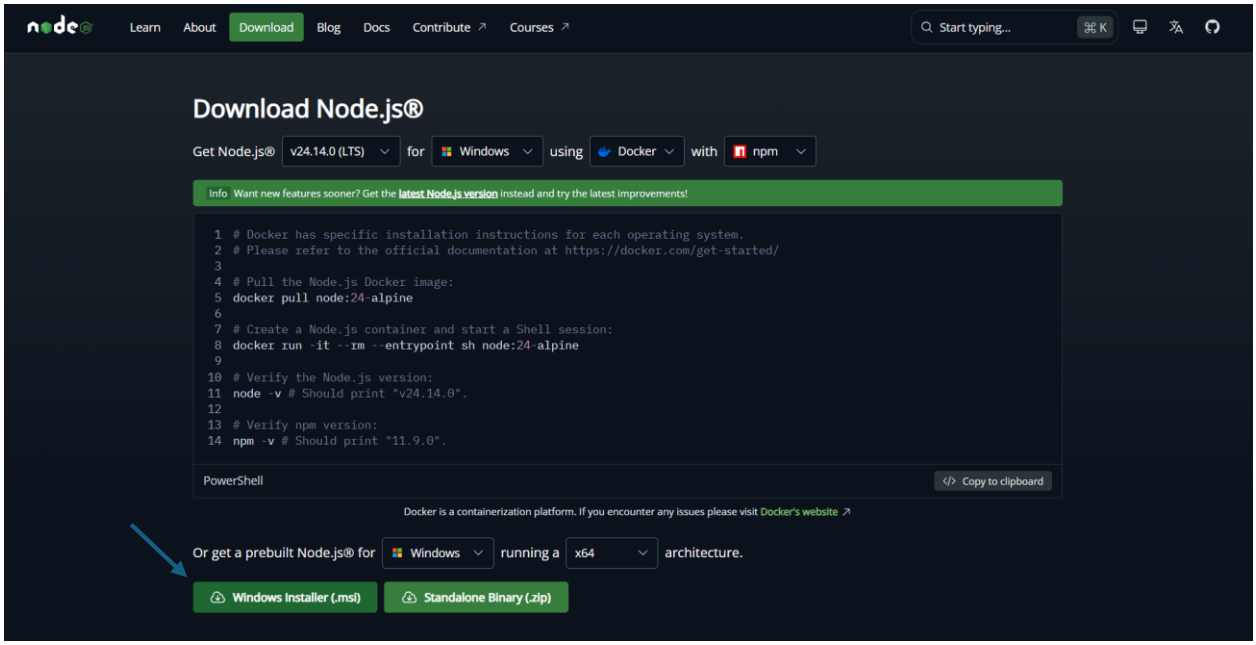
Előfeltételek telepítése:

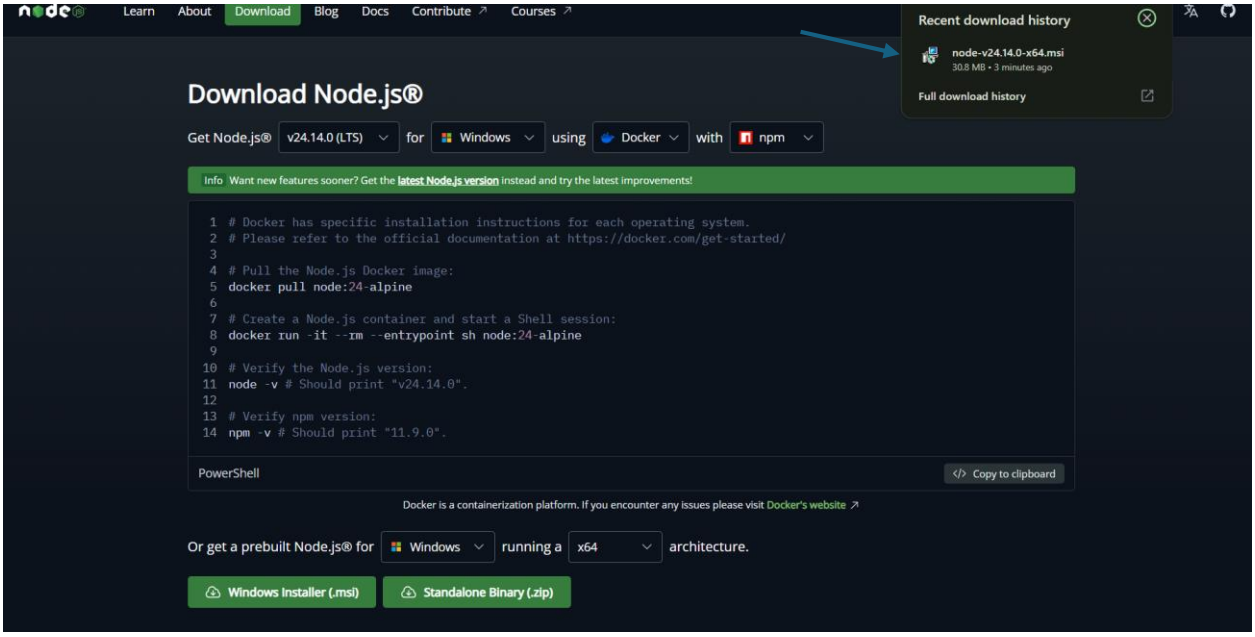
Node.js telepítése:

1. Gondoskodj arról, hogy legyen hálózati elérésed és interneted az eszközödön, lehetőleg számítógép.
2. Nyisd meg a böngésződet.
3. Írd be a keresősávba ezt a linket: https://nodejs.org és nyomj az enterre.
4. Nyomjr rá arra, hogy: “Get Node.js”

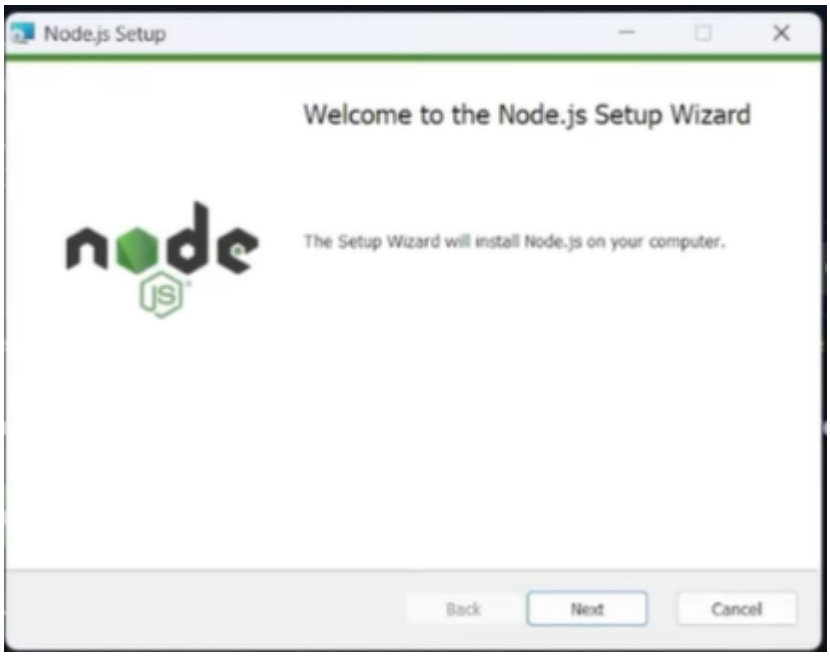



5. Miután átdobott a következő oldalra, kattints a: “Windows Installer (.msi)” gombra és miután letöltött, nyisd meg a fájlt.






6. Miután megnyitottad erre a felületre kell dobnia:

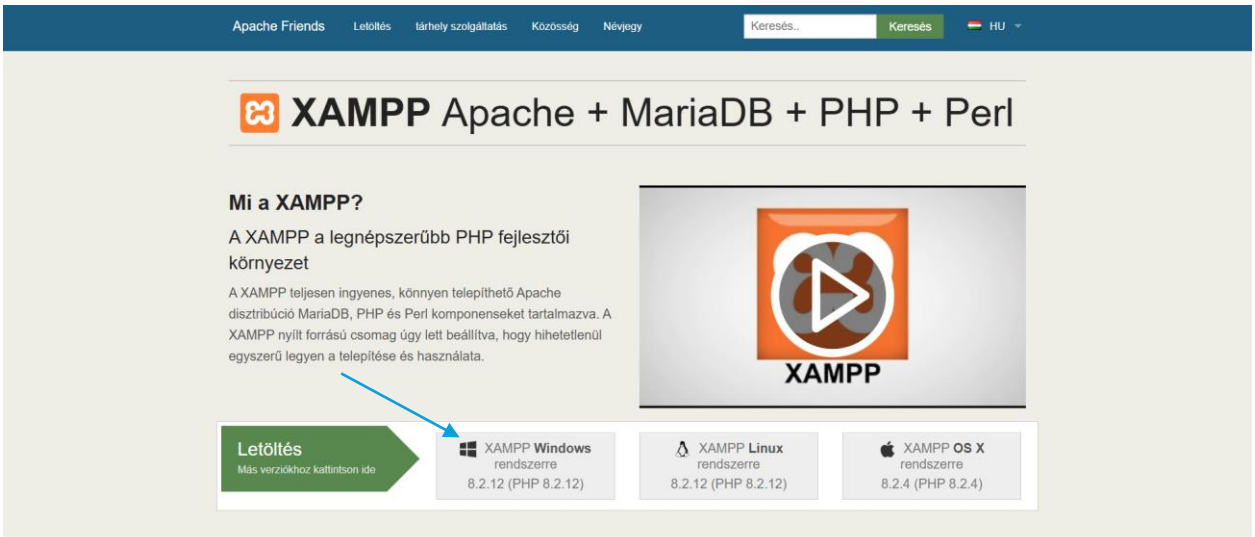



7. Itt rányomsz a “next”-re, elfogadod a végfelhasználói feltételeket, kiválasztod hova szeretnéd tölteni (amit alapból add meg az általában tökéletes), a custom setup résznél rá nyomunk a “Next”-re, a Tools For Native Modules résznél is rányomunk a “Next”-re és a Ready to install Node.js résznél rányomunk az: “Install” gombra és mevárjuk míg letölt. Ezután bezárhatjuk a felületet.

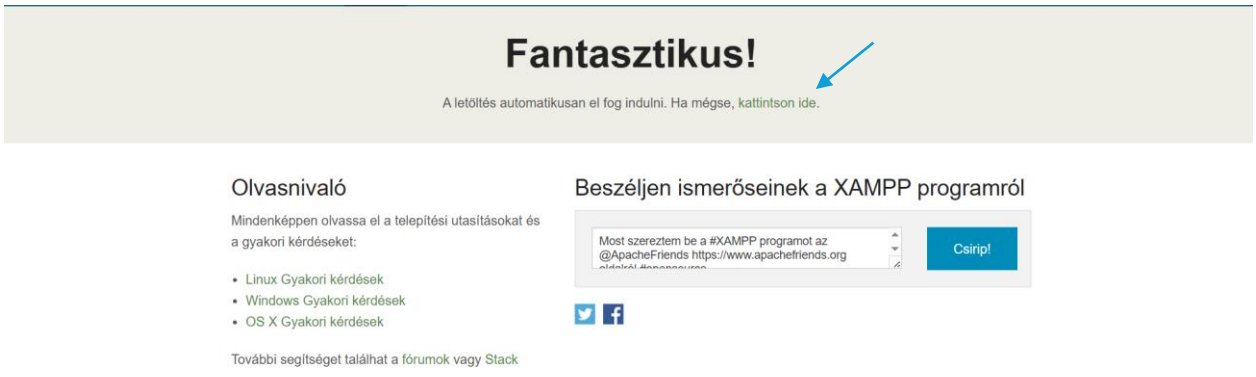
8. Telepítsd a MariaDB-t: https://mariadb.org/download/

## 25.2 XAMPP telepítés:

1. Gondoskodj arról, hogy legyen hálózati elérésed és interneted az eszközödön, lehetőleg számítógép.
2. Nyisd meg a böngésződet.
3. Írd a keresősávba ezt a linket: https://www.apachefriends.org/hu/index.html.
4. Nyomj entert.
5. Az alábbi felületen menj a képen látható Windows lehetőségnél a "XAMPP Windows rendszerre" gombra.

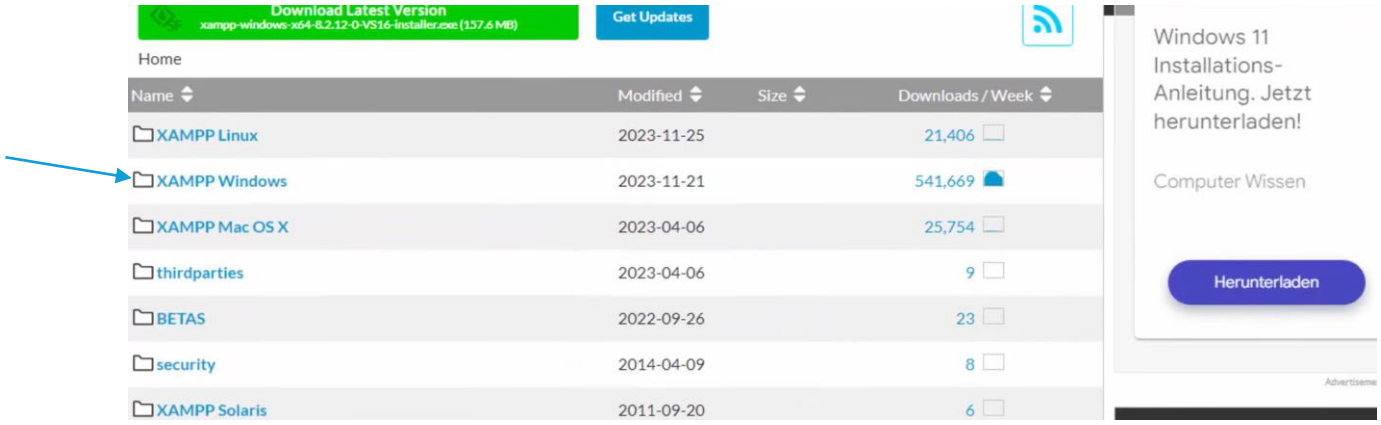


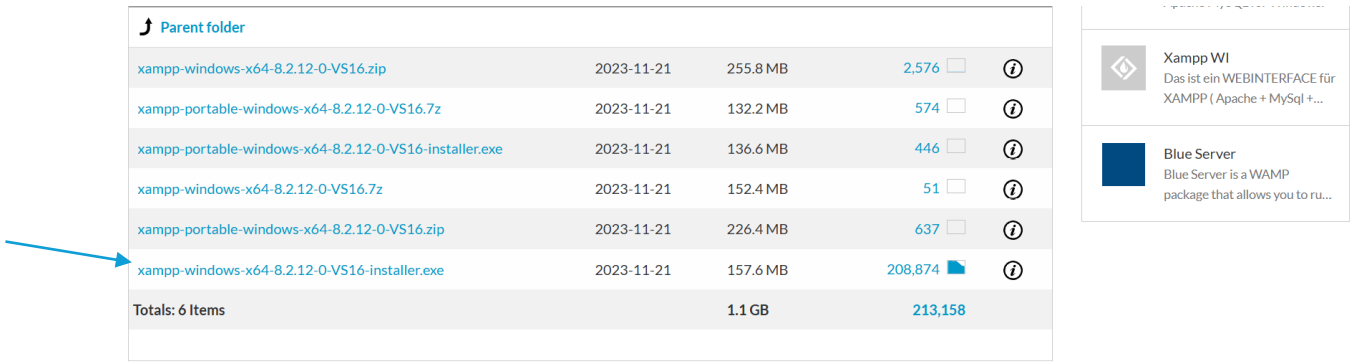

6. Itt átdob egy másik részére az oldalnak, ahol közli, hogy sikeres a letöltés. Ha valamiért nem indult el a letöltés, a "kattintson ide" gombra kell nyomni.



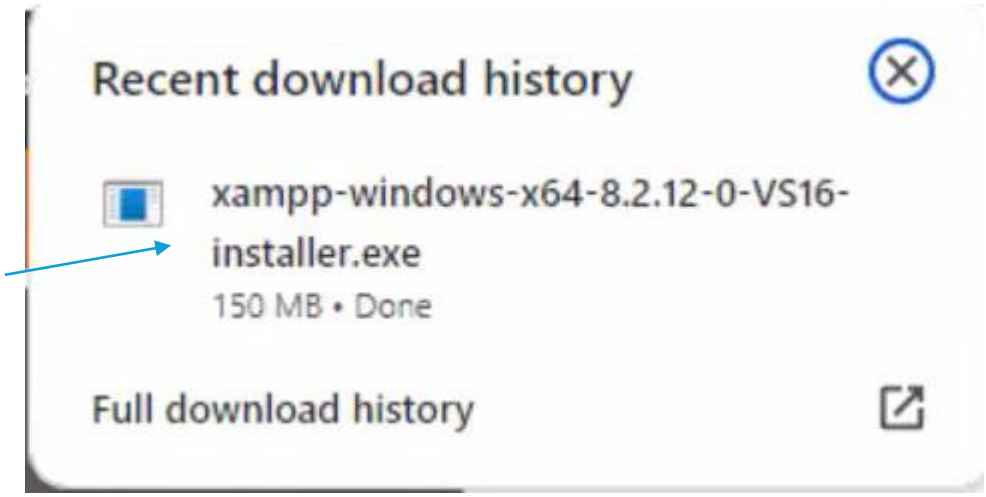

7. Ha letöltött elsőre, nem kell ezzel a lépéssel foglalkoznod, ha nem, akkor kövess minden lépést. Amikor rányomtál a "kattintson ide" gombra megjelenik e az oldal, itt rá kell nyomni a "XAMPP Windows" mappa ikonos gombra. Innen átdob a

verziókhoz és a 8.2.12 mappás gombra kell nyomni. Ez után átdob az összes jelenleg elérhető verziókhoz, amelyek abban a verzió számban találhatók, neked a: [xampp-windows-x64-8.2.12-0-VS16-installer.exe]() -re less szükséged.






8. Miután letöltődik nyisd meg a fájlt.




9. A bejövő ablakban megtalálod a szoftver változtatásait a gépeden, ezt elk ell fogadni.
10. Ez után megnyílik a XAMPP telepítő rendszere. Itt rányomsz a “next” gombra. A komponenseknél amik előre kivannak jelölve tökéletesek, tehát nyomhatjuk a “next” gombot. Ez után ki választja nekünk autómatikusan a XAMPP mappát, ami nekünk tökéletes, tehát “next” gombra nyomunk. Majd kiválasztjuk milyen nyelven szeretnénk használni a felületet, itt az egyszerűség kedvéért válasszuk az angolt, majd “next” gombot kell nyomni. Ez után letölti, és a végén megkérdi, hogy az Apache HTTP Server-nek adunk-e hozzáférést, amire “igent” kell nyomni.
11. A telepítés után ellenőrizd a szolgáltatást: Get-Service -Name "mariadb" vagy a hozzá tartozó szolgáltatást a Services MMC-ben.

## 25.3 Adatbázis létrehozása és jogosultságok (PowerShell vagy XAMPP):

mysql -u root -p

CREATE DATABASE barbershopdb CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

CREATE USER 'barber_user'@'localhost' IDENTIFIED BY 's3cretP@ss';

GRANT ALL PRIVILEGES ON barbershopdb.* TO 'barber_user'@'localhost';

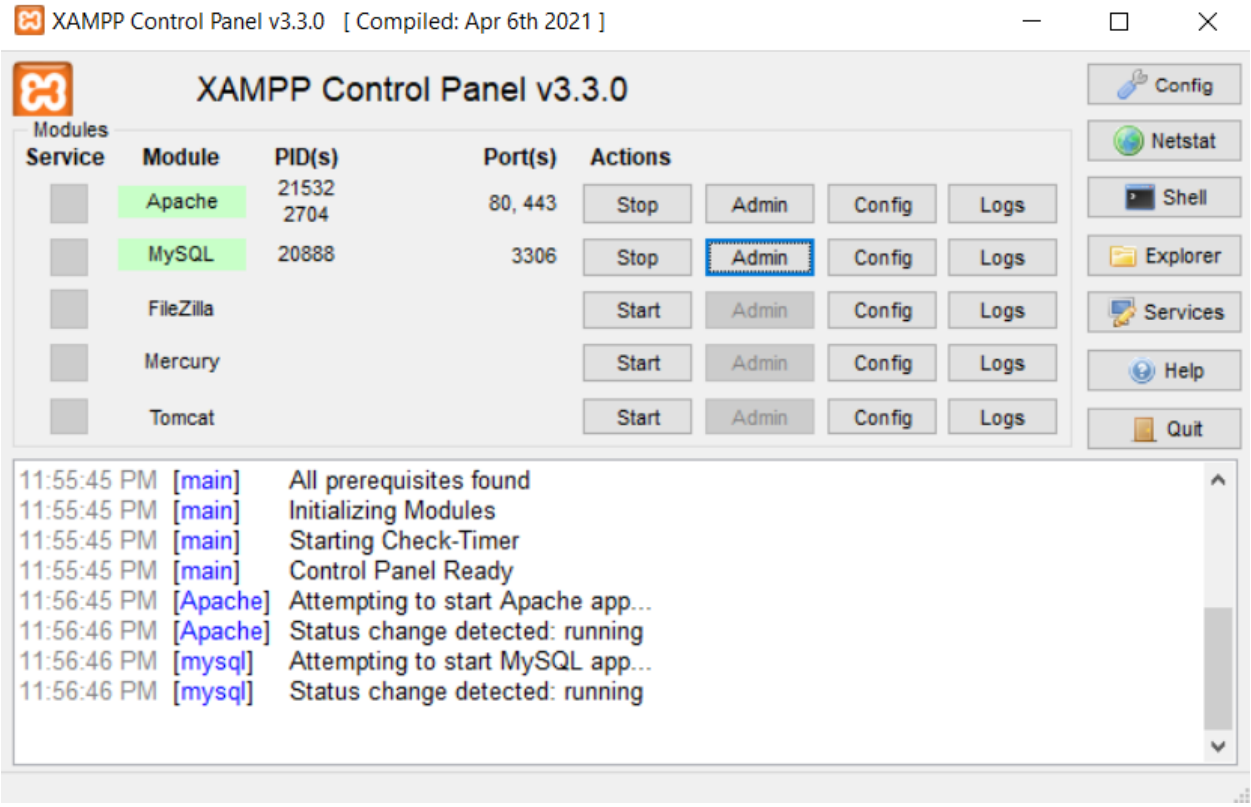
FLUSH PRIVILEGES;

EXIT;

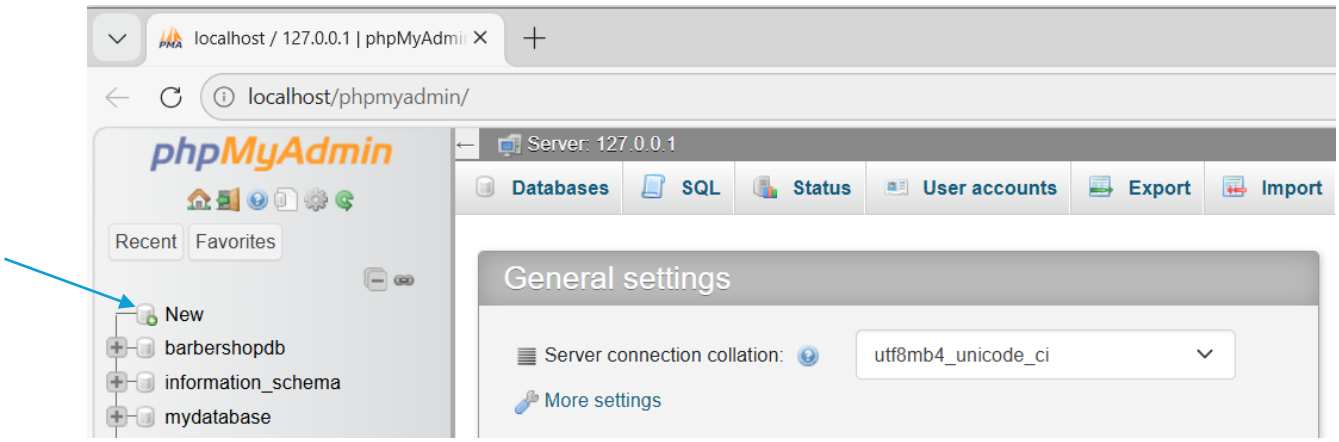
## Adatbázis létrehozása a GUI felületen:

1. Nyisd meg a XAMPP-ot.
2. Indítsd el az Apache-ot.

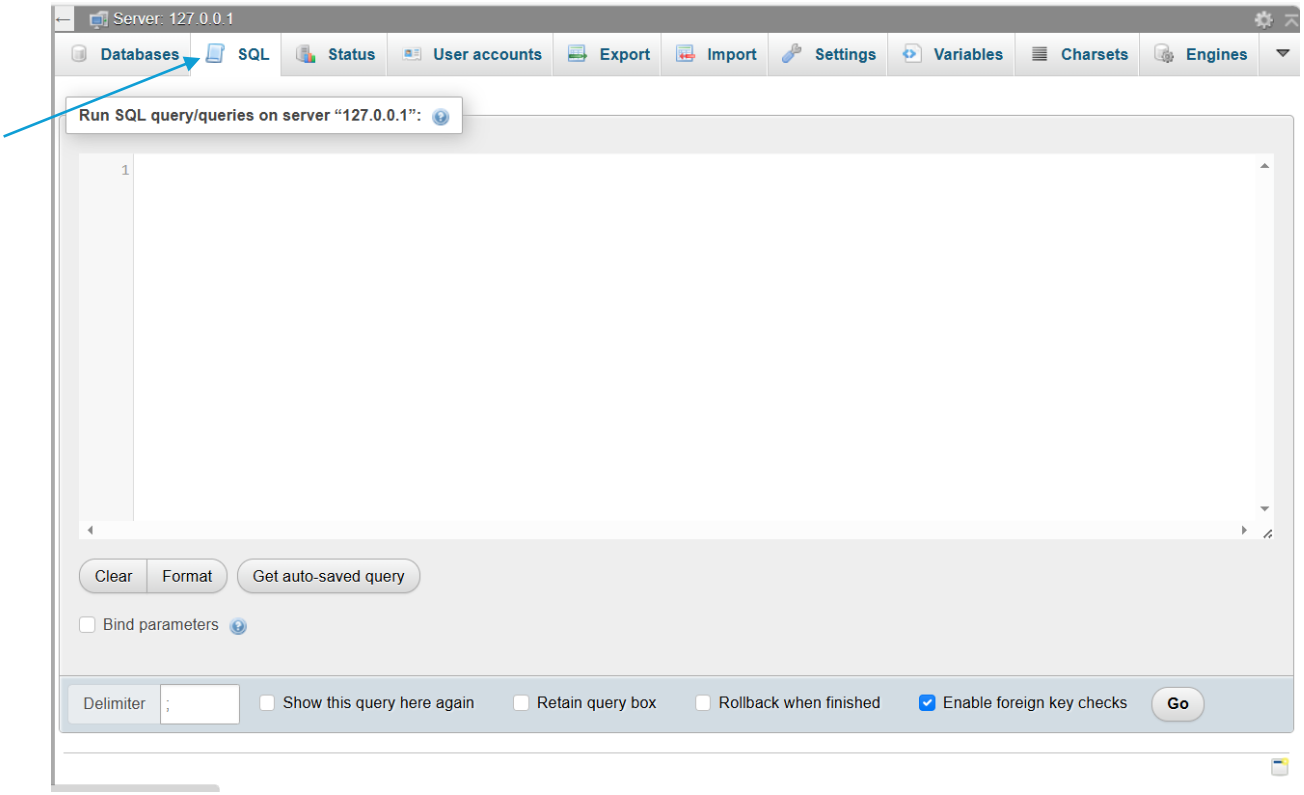
3. Indítsd el a MySQL-t.
4. A MySQL-nél nyomj az Admin gombra, amely átvissz a GUI felületre.




5. Itt a “new” gombra nyomva elkezdjük az adatbázis létre hozását.




6. Az adatbázis neve: barbershopdb, a collation-hoz ezt kell választani: utf8mb4_hungarian_ci, ezek után rányomunk a create gombra.
7. Végül ezt a kódot másoljuk be a SQL felületen:




```
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nev VARCHAR(255) NOT NULL UNIQUE,
  email VARCHAR(255) NOT NULL UNIQUE,
  jelszo VARCHAR(255) NOT NULL,
  foglaltsag ENUM(‘Fogyasztó’, ‘Borbély’, 'Admin') DEFAULT 'Fogyasztó',
  pfPicture VARCHAR(255)
);

CREATE TABLE termekek (
  id INT AUTO_INCREMENT PRIMARY KEY,
  termekNev VARCHAR(255) NOT NULL,
  megjegyzes VARCHAR(255),
  ar DECIMAL(10,2) NOT NULL DEFAULT 0.00,
```

```
  kepNeve VARCHAR(255),
  kategoria VARCHAR(255)
);

-- A rendelés és rendeles_termekek kapcsolatai
CREATE TABLE rendelesek (
    id INT(255) PRIMARY KEY AUTO_INCREMENT,
    vasarloNeve VARCHAR(75) NOT NULL,
    vasarloEmail VARCHAR(75) NOT NULL,
    telefonszam VARCHAR(12),
    iranyitoszam VARCHAR(4) NOT NULL,
    telepules VARCHAR(255) NOT NULL,
    szallitasiCim VARCHAR(255) NOT NULL,
    rendelesIdeje DATETIME NOT NULL,
    termekek JSON NOT NULL,
    ar INT(255) NOT NULL,
    userId INT,
    FOREIGN KEY (userId) REFERENCES users(id) ON UPDATE CASCADE ON
DELETE CASCADE
);

CREATE TABLE rendelesTermekek (
  id INT AUTO_INCREMENT PRIMARY KEY,
  rendeles_id INT NOT NULL,
  termek_id INT NOT NULL,
  mennyiseg INT NOT NULL,
  ar DECIMAL(10,2) NOT NULL,
```

```
  FOREIGN KEY(rendeles_id) REFERENCES Rendelesek(id) ON DELETE CASCADE,
  FOREIGN KEY(termek_id) REFERENCES Termekek(id) ON DELETE CASCADE
);
CREATE TABLE foglalasok (
  id INT(255) PRIMARY KEY AUTO_INCREMENT,
  vasarloNeve VARCHAR(75) NOT NULL,
  vasarloEmail VARCHAR(255) NOT NULL,
  vasarloTelefonszam VARCHAR(12) NOT NULL,
  idopont DATETIME NOT NULL,
  borbely VARCHAR(75) NOT NULL,
  szolgaltatas VARCHAR(255) NOT NULL,
  ar INT(255) NOT NULL,
  userId INT,
  borbelyId INT,
  FOREIGN KEY (userId) REFERENCES users(id) ON UPDATE CASCADE ON DELETE SET NULL,
  FOREIGN KEY (borbelyId) REFERENCES users(id) ON UPDATE CASCADE ON DELETE SET NULL
);
```

## 25.4 Projekt klónozása és függőségek telepítése:

1. Nyisd meg a böngésződ. Ezen az útvonalon keresd meg azt a felhasználót, akinél tárolni fogod a projektet, általában így nézz ki: Local Disk>Users><az adott felhasználó>.
2. Itt hozz létre egy mappát a job klik segítségével és menj rá a “folder” vagy “mappa” gombra, nyelvtől függően. Az új mappa neve legyen “git”.
3. Nyisd meg a Visual Studio Code-ot és nyiss meg egy új terminált.
4. A terminálba írd be ezt a kód sort: git clone https://github.com/Szatocs1/Final_Project és nyomj entert. Ez után a project klónozva van.
5. Most be kell lépned ebbe a projektbe. A Visual Studio Code file gombra nyomva egy lenyíló lista kerül elő, itt az: “open folder”-t kell nyomni. Menj a “This PC”-re és

ebből kiindulva keresd meg a git mappát, ezen a vonalon: Local Disk>Users><az adott felhasználó>>git. Kiválasztod a Final_Projectet és megnyitod.

6. Most megjelenik a szerkeztői környezetben a kód.
7. Ezek után az alábbi kódokat másold be így a terminálba:

cd "Final_Project/Barbershop 1.0.6/Backend"

npm install

8. Ezután a gyökér mappába (Backend) hozz létre egy ilyen nevű fájlt: “.env” és az alábbi javasolt adatokkal töltsd ki:

## Környezeti változók létrehozása a .env-ben — javasolt példa:

PORT=3000

DB_HOST=localhost

DB_PORT=3306

DB_USER=root

DB_PASS=null

DB_NAME=barbershopdb

JWT_SECRET= mySuperSecretJWTKey12345678901234567890

JWT_EXPIRES=45m

MAIL_HOST=smtp.gmail.com

MAIL_PORT=587

MAIL_USER= divisum.noreply@gmail.com

MAIL_PASS= arle kbos cfkl drag

MAIL_FROM = divisum.noreply@gmail.com

9. Futtatás fejlesztés alatt (nodemon javasolt), ezeket az indító parancsokat használd az elindítás során:

npx nodemon server.js

Vagy simán:

node server.js

Bekapcsolódó ellenőrzés:

- Teszteld a bejelentkezést és terméklistát Postman-nel.

## 25.5 Adatbázis konfiguráció részletek (Sequelize példával)

Példa config/db.js (Sequelize alap):

```
const { Sequelize, DataTypes } = require('sequelize');

// Initialize Sequelize with MariaDB
const sequelize = new Sequelize({
  dialect: 'mariadb',
  host: process.env.DB_HOST || 'localhost',
  port: process.env.DB_PORT || 3306,
  database: process.env.DB_NAME || 'barbershopdb',
  username: process.env.DB_USER || 'root',
  password: process.env.DB_PASS || '',
  logging: console.log, // Enable logging for debugging
});

// Sync database
async function syncDatabase() {
  try {
    await sequelize.authenticate();
    console.log('Database connection established.');
    await sequelize.sync(); // Create tables if they do not exist, only add {alter: true} if you change the models.
    console.log('Database synced.');
  } catch (error) {
    console.error('Unable to connect to the database:', error);
  }
}
```

```
module.exports = {
  sequelize,
  syncDatabase
};
```

Ellenőrzés a csatlakozásra:

```
db.syncDatabase().then(async ()=>{
   console.log("Adatbázis szinkronizálva!")
   app.listen(PORT, () => {
   console.log(`Szerver fut: http://localhost:${PORT}`);
});
})
```

## 25.6 Migrációk, seedek és adatbázis inicializálás

Ha nincs CLI, készíts egy startup scriptet, ami ellenőrzi, hogy léteznek-e a táblák, és szükség esetén létrehozza őket vagy lefuttatja a seedeket.

Ha van CLI és azzal szeretnénk csinálni:

Sequelize CLI használata (ha projectben telepítve):

```
npx sequelize-cli db:migrate
npx sequelize-cli db:seed:all
```

## 25. Backup és Restore (parancsok)

Full dump (mysqldump):

```
mysqldump -u root -p barbershopdb > barbershopdb_$(date +%F).sql
```

Restore:

```
mysql -u root -p barbershopdb < barbershopdb_2026-03-03.sql
```

## 26. Hálózat, tűzfal és portok

- Alapértelmezett DB port: 3306. Ha az adatbázist csak a szerver számára engedélyezett, akkor ne nyisd ki a 3306-ot a publikus internet felé.
- Engedélyezd a 3000 portot a reverse proxy felé, vagy csak a 80/443 portot nyisd a külvilág felé, a node process pedig localhoston fusson.

## 27. Hibakeresés — gyakori problémák és megoldások

- ER_ACCESS_DENIED_ERROR: ellenőrizd a DB user/jelszó beállítást a .env-ben és hogy a felhasználó jogosultsággal rendelkezik a DB-hez.
- ECONNREFUSED DB kapcsolatnál: ellenőrizd, hogy a MariaDB fut és a host/port helyes.
- MODULE_NOT_FOUND vagy Cannot find module 'express': futtasd az npm install-t a projekt gyökérben.
- 500 hibák: nézd a szerver logot (console output vagy pm2 logs) a részletekért.

4. Részletes üzemeltetési lépések

## 28. Naplózás és monitoring

- Telepíts strukturált loggert (Winston/Pino) és küldd a fontos logokat egy centralizált rendszerbe (pl. ELK stack vagy LogDNA).
- Monitoring: állíts be metrikagyűjtőt (Prometheus) és dashboardokat (Grafana).

## 29. Frissítések

- Rendszeresen futtass npm audit-ot és frissíts csomagokat. Teszteld a frissítést staging környezetben, mielőtt productionbe küldenéd.

## 30. Biztonsági mentés és helyreállítás tesztelése

- Legalább havonta teszteld a restore folyamatot egy elkülönített környezetben.

## 31. Mellékletek és példa fájlok

- Példa config/db.js (Sequelize) fent.
- Példa .env felsorolva az install résznél.

## 32. Részletes környezeti változó-gyűjtemény

A rendszer különböző részei rengeteg környezeti változóra támaszkodnak; az alábbi lista segít áttekinteni őket és megérteni, mire szolgálnak. Minden változó, ha nem adunk meg értéket – hibához vezethet, ezért felhasználás előtt ellenőrizzük őket, hogy a helyes értékek vannak-e megadva.

- PORT – az Express szerver hallgató portja (általában 3000 vagy 8080). Ha ütközés van, módosítsd.
- DB_HOST – adatbázis gép címe (localhost fejlesztéshez, távoli hoszt a termeléshez).
- DB_PORT – adatbázis port (alap 3306 MariaDB/MySQL esetén).
- DB_USER – adatbázis felhasználó neve.
- DB_PASS – adatbázis jelszó.
- DB_NAME – használni kívánt adatbázis neve.
- JWT_SECRET – a JWT aláíró titok; legalább 32 karakter hosszú, véletlenszerű.
- JWT_EXPIRES – a token élettartama (1h, 7d stb.).
- MAIL_HOST – SMTP szerver címe e-mail küldéshez.
- MAIL_PORT – SMTP port (25, 465, 587).
- MAIL_USER – SMTP felhasználónév.
- MAIL_PASS – SMTP jelszó.

### Ezen kívül opcionális beállítások:

- LOG_LEVEL – logolás szintje (debug, info, warn, error).
- UPLOAD_DIR – alap mappa fájlfeltöltések számára, pl. uploads/.
- NODE_ENV – környezet (development, production, test); határozza meg az Express hibakezelés és logolás viselkedését.
- CACHE_TTL – cache élettartam milliszekundumban.
- RATE_LIMIT – egy felhasználó által egységnyi idő alatt engedélyezett kérések száma.
- SMTP_SECURE – true/false a TLS használatához.
- SESSION_SECRET – ha session alapján is tárolunk cookie-kat.

Az .env.example fájl tartalmazza a kötelező kulcsok mintáit. A változók lehetnek külön .env fájlban (helyi fejlesztés) vagy CI/CD rendszerben/host környezetben (pl. Azure App Service, Docker Compose environment). Tartsuk titokban a jelszavakat: ne kerüljön semmi belőlük verziókezetéssel.

A változók bővülhetnek további modulokkal (pl. külső API kulcsok, Google Captcha, külső tárhely stb.), ezért vegyük figyelembe, hogy bármikor dokumentálni kell minden új beállítást.

## 33. Záró megjegyzések

Ez a dokumentáció a rendszer jelenlegi állapotához nyújt referenciát és ajánlásokat. A pontos részletek (mezők, típusok) attól függnek, milyen ORM/driver és milyen üzleti követelmények kerülnek véglegesítésre. Javasolt a dokumentáció verziózását karban tartani, és minden séma módosítás után migrációs tervet és frissített ERD-t készíteni.

## 34. Végszó

Ez a kiterjesztett útmutató részletes lépéseket ad a Backend és az adatbázis telepítéséhez és üzemeltetéséhez fejlesztési, Docker és production környezetekben. Ha szeretnéd, elkészítem a dokumentum Word (.docx) verzióját, vagy egy rövidített telepítési checklistet külön fájlban.

FRONTEND

# Divisum Frontend Dokumentáció

Ez a dokumentáció részletesen bemutatja a Divisum Angular alapú frontend alkalmazásának felépítését, komponenseit, szolgáltatásait és működését. A dokumentum célja, hogy teljes körű áttekintést nyújtson a fejlesztők számára, és megkönnyítse a projektben való navigációt, a karbantartást és a továbbfejlesztést. A formázás (címsorok, kódblokkok) lehetővé teszi a dokumentum egyszerű átmásolását Word-be.

## 1. Bevezetés

A Divisum egy modern, egyoldalas webalkalmazás (SPA - Single Page Application), amely az Angular keretrendszerre épül. A projekt célja egy reszponzív, felhasználóbarát és könnyen karbantartható felület biztosítása egy fiktív borbélyszalon és webshop számára.

**Főbb funkciók:**

- Felhasználói authentikáció (regisztráció, bejelentkezés, kijelentkezés).
- Adminisztrációs felület a felhasználók, termékek, rendelések és foglalások kezelésére.
- Termékek böngészése és vásárlása.
- Kosár funkcionalitás és rendelésleadás.
- Időpontfoglalás a szalon szolgáltatásaira.
- Felhasználói profiloldal.
- Statikus jogi oldalak (ÁSZF, Adatvédelem).

**Technológiai háttér:**

- **Keretrendszer:** Angular (~v17), amely a komponensalapú architektúrára, a TypeScript nyelvre és a reaktív programozási paradigmákra támaszkodik.
- **Stílus:** SCSS a globális témákhoz és a komponens-specifikus stíluslapokhoz. Az Angular Material komponenskönyvtár és a Material 3 dizájnrendszer adja a felhasználói felület alapját.
- **Állapotkezelés:** A kliensoldali állapotkezelés elsősorban Angular szolgáltatások (Services) és RxJS BehaviorSubject-ok segítségével valósul meg, ami egy könnyű és hatékony megoldás a közepes komplexitású állapotok menedzselésére.
- **HTTP kommunikáció:** Az Angular beépített HttpClient modulja felel a backend API-val való kommunikációért.

## 2. Projektstruktúra

A projekt egy szabványos Angular CLI által generált struktúrát követ, amely logikusan elkülöníti az alkalmazás különböző részeit.

```
Divisum/

├── angular.json          # Angular CLI konfigurációs fájl

├── package.json          # Projekt függőségek és scriptek

├── tsconfig.json         # TypeScript fordító beállításai

├── src/

│   ├── index.html        # A fő HTML fájl

│   ├── main.ts           # Az alkalmazás belépési pontja

│   ├── styles.css        # Globális, nem Angular Material stílusok
```

```
│   ├── custom-theme.scss       # Globális Angular Material téma definíciók
│   ├── app/
│   │   ├── app-module.ts          # Fő alkalmazás modul (ha nem standalone)
│   │   ├── app-routing-module.ts   # Fő útválasztó modul
│   │   ├── app.ts                 # Fő komponens
│   │   ├── home/                  # Főoldal komponens
│   │   ├── login/                 # Bejelentkezés komponens
│   │   ├── register/              # Regisztráció komponens
│   │   ├── profil/                # Felhasználói profil komponens
│   │   ├── admin/                 # Adminisztrációs felület komponense
│   │   ├── termekek/              # Termékeket listázó oldal
│   │   ├── kosar/                 # Kosár komponens
│   │   ├── foglalas/              # Időpontfoglalás komponens
│   │   ├── adatvedelem/           # Adatvédelmi tájékoztató oldala
│   │   ├── aszf/                  # ÁSZF oldal
│   │   └── services/              # Alkalmazás-szintű szolgáltatások (pl. API, auth)
│   └── assets/
│       └── images/                # Képek és egyéb statikus assetek
```

## Konfigurációs és Rendszerfájlok

Ezek a fájlok a projekt gyökerében találhatók, és a fejlesztői környezet, valamint a build folyamat alapjait adják.

- **angular.json**: Az Angular CLI központi konfigurációs fájlja. Meghatározza a projekt felépítését, az eszközök (assets) helyét, és a fordítási beállításokat.

- **package.json**: Tartalmazza a projekthez szükséges összes NPM csomagot (függőségeket) és az automatizálható scripteket (pl. start, build).
- **tsconfig.json**: A TypeScript fordító beállításait tartalmazza, például a modulfeloldási szabályokat és a célnyelvi verziót.
- **main.ts**: Az alkalmazás belépési pontja. Ez a fájl felelős az Angular alkalmazás elindításáért (bootstrap) a böngészőben.
- **index.html**: Az egyetlen tényleges HTML oldal (SPA jelleg), amelybe az Angular beilleszti a komponenseket.

## Globális Stílusok és Erőforrások

Az alkalmazás vizuális megjelenéséért és a statikus médiatartalomért felelős réteg.

- **custom-theme.scss**: Globális Angular Material téma definíciók. Itt kerülnek beállításra a Material 3 specifikáció szerinti színek (primary, tertiary) és a tipográfia .
- **styles.css**: Általános globális stíluslap a nem Material-specifikus formázásokhoz és egyedi betűtípusokhoz.
- **assets/images/**: Ez a mappa tárolja az összes statikus képet, ikont és egyéb médiát, amit az alkalmazás használ .

## Alkalmazásmag és Útvonalválasztás

Az src/app/ mappában található központi vezérlő fájlok.

- **app-module.ts**: A fő alkalmazásmodul, amely összefogja a különböző komponenseket és szolgáltatásokat (ha nem standalone felépítésű az app).
- **app-routing-module.ts**: Meghatározza az alkalmazás útvonalait (pl. /login, /admin), összekapcsolja az URL-eket a komponensekkel, és alkalmazza a hozzáférési korlátozásokat (guards).
- **app.ts**: Az alkalmazás gyökérkomponense, amely vázként szolgál a többi oldal számára.

## Oldal-komponensek (UI Modulok)

Minden mappa egy-egy funkcionális egységet vagy oldalt takar, általában .ts, .html és .css fájlokkal.

- **home/**: A főoldalért felelős komponens. Kezeli a borbélyok listázását és a termékek kiemelését.
- **login/**: A felhasználói bejelentkezési felület és logika.

- **register/**: Az új felhasználók regisztrációját kezelő felület.
- **profil/**: A bejelentkezett felhasználók saját adatlapja; AuthGuard-dal védett útvonal.
- **admin/**: A legösszetettebb, standalone egység. Feladata a felhasználók, termékek, rendelések és foglalások CRUD menedzselése .
- **termekek/**: A webshop termékkínálatának listázó oldala.
- **kosar/**: Standalone komponens a kiválasztott tételek megjelenítésére, mennyiségmódosításra és a rendelés leadására .
- **foglalas/**: A szalon szolgáltatásaira történő időpontfoglalás felülete.
- **adatvedelem/**: Statikus információs oldal az adatvédelmi tájékoztatóval.
- **aszf/**: Az Általános Szerződési Feltételeket tartalmazó oldal.

## Szolgáltatások és Biztonság

A services/ mappa tartalmazza az üzleti logikát és a backenddel való kapcsolatot.

- **kosar.services.ts**: A kosár állapotát kezeli BehaviorSubject segítségével, biztosítva az adatok szinkronizálását a komponensek között .
- **auth.guard.ts**: Útvonalvédő (Route Guard), amely ellenőrzi a felhasználó jogosultságait (pl. token megléte), mielőtt engedélyezné a belépést a védett oldalakra.
- **API Szolgáltatások**: Ide tartoznak azok a logikai egységek, amelyek a HttpClient segítségével kommunikálnak a backend végpontokkal (pl. rendelések küldése, termékek lekérése).

# 3. Főbb Angular komponensek

Az alkalmazás komponensekre van bontva, ahol minden komponens egy-egy UI részletért vagy oldalért felelős. A projekt vegyesen használja a hagyományos, NgModule-ban deklarált és a modernebb, standalone komponenseket.

## 3.1. App modul és routing

Az AppRoutingModule definiálja az alkalmazás fő útvonalait, és összerendeli az URL-eket a megfelelő komponensekkel.

**app-routing-module.ts**

```
import { NgModule } from '@angular/core';

import { RouterModule, Routes } from '@angular/router';
```

```
import { HomeComponent } from './home/home';

import { LoginComponent } from './login/login';

import { ProfilComponent } from './profil/profil';

import { Admin } from './admin/admin'; // Standalone komponens importálása

import { AuthGuard } from './services/auth.guard'; // Route Guard


const routes: Routes = [

  { path: '', component: HomeComponent },

  { path: 'login', component: LoginComponent },

  { path: 'profil', component: ProfilComponent, canActivate: [AuthGuard] }, // Védett útvonal

  { path: 'admin', component: Admin, canActivate: [AuthGuard] }, // Védett útvonal

  // ... további útvonalak

  { path: '**', redirectTo: '' } // Minden más URL a főoldalra irányít

];


@NgModule({

  imports: [RouterModule.forRoot(routes)],

  exports: [RouterModule]

})

export class AppRoutingModule { }
```

A canActivate: [AuthGuard] sor biztosítja, hogy bizonyos útvonalak (pl. /profil, /admin) csak bejelentkezett felhasználók számára legyenek elérhetők.

## 3.2. Oldal-komponensek részletesen

Minden oldal-komponens a saját mappájában helyezkedik el, és általában három fájlból áll: .ts (logika), .html (sablon), és .css (stílus).

*Home komponens (home/home.ts)*

A főoldal felelős a borbélyok, kiemelt termékek és egyéb marketing tartalmak megjelenítéséért.

- **Dinamikus Adatbetöltés:** Az ngOnInit életciklus-horogban hívja meg a loadBarbers metódust, amely egy HTTP GET kéréssel lekéri a borbélyok listáját a backendről.
- **Felhasználói Interakciók:** Kezeli a termékek kosárba helyezését (addToCart), valamint a termék részleteit megjelenítő modális ablak (openDetails) megnyitását és bezárását.
- **Állapotkezelés:** Lokális változókat használ a UI állapotának követésére, mint például showModal (modális ablak láthatósága) vagy showQuickSizeSelect (gyors méretválasztó megjelenítése ruházati termékeknél).
- **Szolgáltatás-interakció:** A KosarService szolgáltatást használja a termékek kosárhoz adásához.

**home.ts - Részlet a kosárkezelésről és modális ablakról**

```
// ... importok



export class Home implements OnInit {

  // ... egyéb változók

  showModal = false;

  selectedProduct: any = null;

  showQuickSizeSelect = false;

  tempProduct: any = null;

  selectedSize: string = 'M';
```

```
  constructor(public kosarService: KosarService, private http: HttpClient, private cdr: ChangeDetectorRef) {}


  ngOnInit(): void {

    this.loadBarbers();

  }


  loadBarbers() {

    this.http.get<any>('http://localhost:3000/api/auth/borbelyok').subscribe({

      next: (res) => {

        this.barbers = res.borbelyok || (Array.isArray(res) ? res : []);

        this.cdr.detectChanges(); // Változásdetektálás manuális indítása

      },

      error: (err) => console.error('Hiba a borbélyok betöltésekor:', err)

    });

  }


  addToCart(product: any) {

    // Ha ruházat és még nincs méret választva, megjelenítjük a méretválasztót

    if (product.category === 'Ruházat' && !this.showModal && !this.showQuickSizeSelect) {

      this.tempProduct = product;

      this.showQuickSizeSelect = true;

      document.body.style.overflow = 'hidden'; // Scroll letiltása
```

```
    return;
  }

  const productToAdd = {
    ...product,
    size: product.category === 'Ruházat' ? this.selectedSize : null
  };

  this.kosarService.addToCart(productToAdd);
  this.closeQuickSelect();
}

openDetails(product: any) {
  this.selectedProduct = product;
  this.selectedSize = 'M';
  this.showModal = true;
  document.body.style.overflow = 'hidden';
}

closeModal() {
  this.showModal = false;
  this.selectedProduct = null;
  if (!this.showQuickSizeSelect) document.body.style.overflow = 'auto';
```

```
  }

 }
```

*Kosár komponens (kosar/kosar.ts)*

Ez a komponens egy standalone komponens, ami azt jelenti, hogy nem kell NgModule-ban deklarálni, és a saját függőségeit maga kezeli. Felelős a kosár tartalmának megjelenítéséért és a rendelés leadásáért.

- **Állapotkezelés:** A KosarService-en keresztül iratkozik fel a kosár tartalmának változásaira.
- **Számított Értékek:** getter-eket használ a végösszeg (subtotal) és a kedvezménnyel csökkentett végösszeg (total) dinamikus kiszámítására.
- **Űrlapkezelés:** Kezeli a szállítási adatokat tartalmazó űrlapot, és validálja azt a rendelés elküldése előtt (isFormValid).
- **Authentikáció:** Ellenőrzi, hogy a felhasználó be van-e jelentkezve (checkLoginStatus), mielőtt engedélyezné a rendelés leadását.
- **API Kommunikáció:** Az onSendRendeles metódus egy HTTP POST kérést küld a backendnek, amely tartalmazza a rendelés adatait (termékek, végösszeg, szállítási cím). A kérés Authorization header-t is tartalmaz a felhasználó azonosítására.

**kosar.ts - Rendelés elküldése**

```
// ... importok



 @Component({

  // ... selector, imports, etc.

 })

 export class Kosar {

  // ... változók

  isLoggedIn: boolean = false;



  constructor(public kosarService: KosarService, private http: HttpClient) {

   this.checkLoginStatus();

  }
```

```
checkLoginStatus() {
  const token = localStorage.getItem('token');
  this.isLoggedIn = !!token;
}

get total() {
  const discounted = this.subtotal * (1 - this.discount);
  return Math.round(discounted);
}

isFormValid(): boolean {
  return !!(
    this.iranyitoszam.trim() &&
    this.telepules.trim() &&
    this.szallitasiCim.trim()
  );
}

onSendRendeles() {
  if (!this.isLoggedIn) {
    alert('A rendeléshez kérlek jelentkezz be!');
    return;
```

```
}

if (!this.isFormValid()) {
  alert('Kérlek töltsd ki az összes kötelező mezőt!');
  return;
}

const rendelesData: any = {
  termekek: this.kosarService.cartItems.map(item => ({
    id: Number(item.id),
    name: item.name,
    mennyiseg: item.quantity,
    ar: item.price
  })),
  ar: this.total,
  iranyitoszam: this.iranyitoszam,
  telepules: this.telepules,
  szallitasiCim: this.szallitasiCim
};

const token = localStorage.getItem('token');
const headers = { 'Authorization': `Bearer ${token}` };
```

```
    this.http.post('http://localhost:3000/api/rendeles/rendelesCreate', rendelesData, { headers })
      .subscribe({
        next: (response: any) => {
          window.alert('Rendelés sikeresen elküldve!');
          this.kosarService.clearCart();
          this.resetForm();
        },
        error: (error) => {
          console.error('Hiba a rendelés küldésekor:', error);
          window.alert('Hiba történt a rendelés elküldésekor!');
        }
      });
    }
  // ...
}
```

*Admin komponens (admin/admin.ts)*

Az alkalmazás legösszetettebb komponense, amely egy teljes adminisztrációs felületet biztosít. Ez is egy standalone komponens. A felelősségei több részre bonthatók:

1. **Felhasználókezelés:** Lehetőség van felhasználók keresésére név vagy e-mail alapján, új felhasználók létrehozására, meglévők adatainak módosítására és törlésére.
2. **Termékkezelés:** Új termékek felvitele (képpel együtt), meglévő termékek keresése, módosítása és törlése. A képfeltöltés FormData objektum segítségével történik.
3. **Rendeléskezelés:** Rendelések keresése név vagy e-mail alapján, részletek megtekintése, állapot módosítása és törlés.
4. **Foglaláskezelés:** Időpontfoglalások keresése, módosítása és törlése.

Minden művelet token-alapú authentikációt igényel, a HTTP kérések header-jében elküldve a Bearer tokent. A komponens nagy mérete és sokrétű logikája miatt a jövőben érdemes lehet

kisebb, specializált alkomponensekre bontani (pl. UserManagementComponent, ProductManagementComponent).

**admin.ts - Termék módosítása képpel**

```
// ...


export class Admin {
  // ...
  isProductEditing: boolean = false;
  selectedFile: File | null = null;
  foundProduct: any = null;


  onFileSelected(event: any) {
    if (event.target.files.length > 0) {
      this.selectedFile = event.target.files[0];
    }
  }


  saveProductChanges(product: any) {
    const token = localStorage.getItem('token');
    const headers = new HttpHeaders().set('Authorization', `Bearer ${token}`);


    const formData = new FormData();
    formData.append('id', product.id.toString());
    formData.append('name', product.termekNev || '');
```

```
    formData.append('category', product.kategoria || '');

    formData.append('price', (product.ar || 0).toString());

    formData.append('comment', product.megjegyzes || '');


    // Ha a felhasználó új fájlt választott, azt is hozzáadjuk

    if (this.selectedFile) {

      formData.append('file', this.selectedFile);

    }


    this.http.put<any>('http://localhost:3000/api/termek/admin/productsModify', formData,
{ headers }).subscribe({

      next: () => {

        alert("Termék frissítve!");

        this.isProductEditing = false;

        this.getProductByName(); // Frissítjük a nézetet

      },

      error: (err) => alert("Hiba a módosításkor!")

    });

  }


  onDeleteProduct(productId: number) {

    if (!confirm("Törli a terméket?")) return;

    const token = localStorage.getItem('token');

    const headers = new HttpHeaders().set('Authorization', `Bearer ${token}`);
```

```
  // Figyelem: A backend API itt POST-ot használ törlésre ID-vel a body-ban, de a RESTful gyakorlat a DELETE lenne.

  this.http.post('http://localhost:3000/api/termek/admin/productDelete', { id: productId }, { headers }).subscribe({

    next: () => {

      alert("Termék törölve!");

      this.foundProduct = null;

    }

  });

 }

 // ...

}
```

## 3.3. Szolgáltatások (Services)

A szolgáltatások felelősek az üzleti logikáért, az API kommunikációért és az állapotmegosztásért a komponensek között.

*Kosár Szolgáltatás (kosar.services.ts)*

Ez a szolgáltatás kezeli a kosár állapotát. A BehaviorSubject segítségével egy "forró" observable-t hoz létre, amely mindig a legutolsó értéket tárolja, és azonnal továbbítja azt az új feliratkozóknak.

- **items$:** Egy Observable, amelyre a komponensek (pl. KosarComponent, NavbarComponent) feliratkozhatnak, hogy értesüljenek a kosár tartalmának változásairól.
- **addItem(item):** Hozzáad egy új terméket a kosárhoz. Az immutabilitás megőrzése érdekében a ... (spread) operátorral hoz létre új tömböt.
- **removeItem(index):** Eltávolít egy terméket. A slice() metódussal hoz létre egy másolatot a tömbről a módosítás előtt.
- **updateQuantity(name, amount, size):** Módosítja egy termék mennyiségét.
- **clearCart():** Kiüríti a kosarat (pl. sikeres rendelés után).

**kosar.services.ts - Részlet**

```
import { Injectable } from '@angular/core';
```

```
import { BehaviorSubject } from 'rxjs';


@Injectable({ providedIn: 'root' })
export class KosarService {
  // A BehaviorSubject tárolja a kosár aktuális állapotát (termékek tömbje)
  private cartItemsSource = new BehaviorSubject<any[]>([]);


  // A komponensek erre az Observable-re iratkozhatnak fel
  cartItems$ = this.cartItemsSource.asObservable();


  // Getter a jelenlegi érték szinkron eléréséhez
  get cartItems(): any[] {
    return this.cartItemsSource.value;
  }


  addToCart(product: any) {
    const currentItems = this.cartItems;
    const existingItemIndex = currentItems.findIndex(item => item.id === product.id &&
item.size === product.size);


    if (existingItemIndex > -1) {
      // Ha a termék már a kosárban van, növeljük a mennyiségét
      currentItems[existingItemIndex].quantity += 1;
      this.cartItemsSource.next([...currentItems]);
```

```
    } else {
      // Ha új termék, hozzáadjuk 1-es mennyiséggel
      const newItem = { ...product, quantity: 1 };
      this.cartItemsSource.next([...currentItems, newItem]);
    }
  }

  // ... removeItem, updateQuantity, clearCart metódusok
}
```

## 4. Stílusok és assetek

Az alkalmazás vizuális megjelenését egy többrétegű stílusrendszer határozza meg.

- **src/custom-theme.scss:** Ez a fájl definiálja az Angular Material komponensek globális témáját a Material 3 specifikáció szerint. Itt vannak beállítva az elsődleges (primary) és harmadlagos (tertiary) színek, a tipográfia és a komponensek sűrűsége. A Sass mixinek CSS változókat generálnak, amelyeket az egész alkalmazás használ.

**custom-theme.scss**

```
@use '@angular/material' as mat;

html {
  @include mat.theme((
    color: (
      primary: mat.$azure-palette,
      tertiary: mat.$blue-palette,
    ),
```

```
    typography: Roboto,
    density: 0,
  ));
}


body {
  color-scheme: light; // Alapértelmezett világos téma
  background-color: var(--mat-sys-surface);
  color: var(--mat-sys-on-surface);
  font: var(--mat-sys-body-medium);
  margin: 0;
}
```

- **src/styles.css:** Itt kapnak helyet azok a globális stílusok, amelyek nem specifikusak az Angular Materialra, például egyedi betűtípusok importálása vagy általános HTML elemek (pl. body, a) alapstílusai.
- **Komponens-specifikus .css fájlok:** Minden komponensnek saját stíluslapja van (pl. login/login.css). Az Angular alapértelmezés szerint View Encapsulation-t használ, ami azt jelenti, hogy egy komponens stílusai nem "szivárognak ki" és nem befolyásolnak más komponenseket. Ez megakadályozza a stílusütközéseket és modularizálja a CSS-t.

**Példa: adatvedelem/adatvedelem.css** Ez a stíluslap jól mutatja, hogyan lehet egyedi, konzisztens megjelenést kialakítani egy adott oldaltípushoz (pl. jogi oldalak).

```
.legal-hero {
  background-color: #132435;
  padding: 100px 20px;
  text-align: center;
  color: white;
```

```
}

.legal-section h2 {
  color: #132435;
  font-size: 1.8rem;
  font-weight: 800;
  padding-left: 20px;
  border-left: 4px solid #132435;
  margin-bottom: 25px;
}

.legal-section p {
  color: #333;
  line-height: 1.8;
  font-size: 1.1rem;
}
```

- **src/assets/:** Ez a mappa tartalmazza az összes statikus fájlt, mint például képeket, ikonokat és betűtípusokat, amelyek az alkalmazásban felhasználásra kerülnek.

# 5. Build és futtatás

A projekt életciklusát az npm scriptek kezelik, amelyek a package.json fájlban vannak definiálva.

1. **Függőségek telepítése:** A projekt klónozása után az első lépés a node_modules mappa létrehozása a package.json-ban listázott függőségek telepítésével.

unfold_lessbash
content_copyterminal

```
npm install
```

2. **Fejlesztői szerver indítása:** Ez a parancs elindítja az Angular beépített fejlesztői szerverét (alapértelmezetten a http://localhost:4200/ címen), amely figyeli a fájlváltozásokat és automatikusan újrafordítja a kódot (Hot Module Replacement).

   unfold_lessbash
   content_copyterminal

   ```
   npm start
   ```

3. **Production build készítése:** Ez a parancs lefordítja és optimalizálja az alkalmazást éles környezetre. A folyamat magában foglalja a kód minimalizálását (minification), a felesleges kód eltávolítását (tree-shaking) és a fordítási idő optimalizálását (Ahead-of-Time compilation). A kimeneti fájlok a dist/ mappába kerülnek.

   unfold_lessbash
   content_copyterminal

   ```
   npm run build -- --configuration production
   ```

# 6. Továbbfejlesztési javaslatok

A jelenlegi alkalmazás egy szilárd alapot képez, de számos területen továbbfejleszthető a skálázhatóság, teljesítmény és karbantarthatóság javítása érdekében.

- **Lazy Loading (Lusta betöltés):** Jelenleg az alkalmazás indulásakor minden komponens kódja betöltődik. A nagyobb, nem mindig szükséges modulok (pl. AdminModule, ProfilModule) lusta betöltésével jelentősen csökkenthető a kezdeti betöltési idő. Ezt az útválasztó konfigurációjában a loadChildren tulajdonsággal lehet megvalósítani.

  ```
  // Példa a lazy loadingra az app-routing-module.ts-ben

  const routes: Routes = [

    // ...

    {

      path: 'admin',
  ```

```
    loadChildren: () => import('./admin/admin.routes').then(m => m.ADMIN_ROUTES),

    canActivate: [AuthGuard]

  },

  // ...

];
```

- **Fejlettebb állapotkezelés (pl. NGRX):** Bár a BehaviorSubject-alapú szolgáltatások jól működnek, egy nagyobb, komplexebb alkalmazás esetén egy dedikált állapotkezelő könyvtár, mint az NGRX, központosított és kiszámíthatóbb állapotkezelést biztosíthat. Az NGRX a Redux mintát valósítja meg, ami segít a komplex állapotváltozások nyomon követésében és debuggolásában.
- **Egységes hibakezelés:** Egy HttpInterceptor bevezetésével központosítottan lehetne kezelni a HTTP hibákat (pl. 401 Unauthorized, 500 Internal Server Error), ahelyett, hogy minden egyes subscribe hívás error blokkjában kellene ezt megtenni.
- **Refaktorálás és komponensekre bontás:** Az Admin komponens jelenleg túl nagy és túl sok felelőssége van (SRP sértés). Érdemes lenne kisebb, egyetlen feladatra fókuszáló alkomponensekre bontani (pl. UserTableComponent, ProductEditorComponent), ami javítaná az olvashatóságot és az újrafelhasználhatóságot.
- **CI/CD (Continuous Integration/Continuous Deployment) pipeline:** Egy automatizált pipeline beállítása (pl. GitHub Actions, GitLab CI) biztosíthatná, hogy minden kódfeltöltés után automatikusan lefusson a linter, a tesztek és a build folyamat, ezzel garantálva a kódminőséget és csökkentve a manuális hibák lehetőségét.

# 35. Források és dokumentáció

A projekt fejlesztése során számos online forrást lehet hasznosítani. Az alábbiak a felhasználható linkek gyűjteménye – az általad felsoroltak mellett néhány további, jól bevált dokumentáció és tutorial is szerepel, amelyek hasznosak lehetnek a Node.js/Express/Sequelize/JWT stack használatához.

- https://sequelize.org/
- https://expressjs.com/
- https://nodejs.org/en/docs/
- https://jwt.io/introduction/
- https://www.npmjs.com/package/bcrypt
- https://www.npmjs.com/package/jsonwebtoken
- https://mariadb.com/kb/en/
- https://www.mysql.com/
- https://www.npmjs.com/package/dotenv
- https://www.npmjs.com/package/express-session
- https://jasonwatmore.com/post/2020/09/08/nodejs-mysql-boilerplate-api-with-email-sign-up-authentication
- https://jasonwatmore.com/post/2020/08/18/nodejs-mysql-simple-api-for-authentication-registration-and-user-management
- https://www.bezkoder.com/node-js-express-login-example/
- https://www.bezkoder.com/node-js-jwt-authentication-mysql/
- https://www.bezkoder.com/react-express-authentication-jwt/
- https://www.bezkoder.com/node-express-vue-jwt-auth/
- https://www.corbado.com/blog/nodejs-express-mysql-jwt-authentication-roles
- https://www.djamware.com/post/6158fee27523f53fb5c1b2f4/authentication-role-permission-api-using-node-express-mysql
- https://github.com/indraarianggi/nodejs-sequelize-mysql-api
- https://github.com/bezkoder/node-js-jwt-auth
- https://github.com/jmw5598/node-express-jwt-example

- https://github.com/MarkKhramko/nodejs-express-jwt
- https://gist.github.com/ansthsys/53400202b0422d88f85011eda90a5ee4
- https://jwt.io/
- https://stackabuse.com/understanding-json-web-tokens-with-node-js/
- https://dev.to/azure/secure-your-nodejs-api-with-json-web-tokens-jwt-5a5g
- https://blog.logrocket.com/node-authentication-jwt-bcrypt/
- https://medium.com/@eminetto/authentication-in-nodejs-beyond-jwt-483fb6809f8c
- https://sequelize.org/master/manual/getting-started.html
- https://mariadb.com/resources/blog/getting-started-with-mariadb-and-nodejs/
- https://getcssscan.com/css-box-shadow-examples


*További javasolt források:*


- MDN Web Docs (Express és Node.js API referenciák): https://developer.mozilla.org/en-US/docs/Learn/Server-side/Express_Nodejs
- Postman hivatalos dokumentáció és gyűjtemények: https://www.postman.com/
- OpenAPI/Swagger felület tervezéshez: https://swagger.io/
- RFC 7519 – JSON Web Token (alapszabvány) https://tools.ietf.org/html/rfc7519
- Sequelize v6 docs: https://sequelize.org/docs/v6/